Aula 02 – Conjunto de instruções

Prof. João Fernando Mari
joaofmari.github.io

Roteiro

- Introdução
- Operações no hardware do computador
- Operandos do hardware do computador
- Constantes ou operandos imediatos
- Representando instruções no computador
- Operações lógicas
- Instruções para tomada de decisões
- Suporte para procedimentos no hardware do computador
- Comunicando-se com as pessoas

Introdução

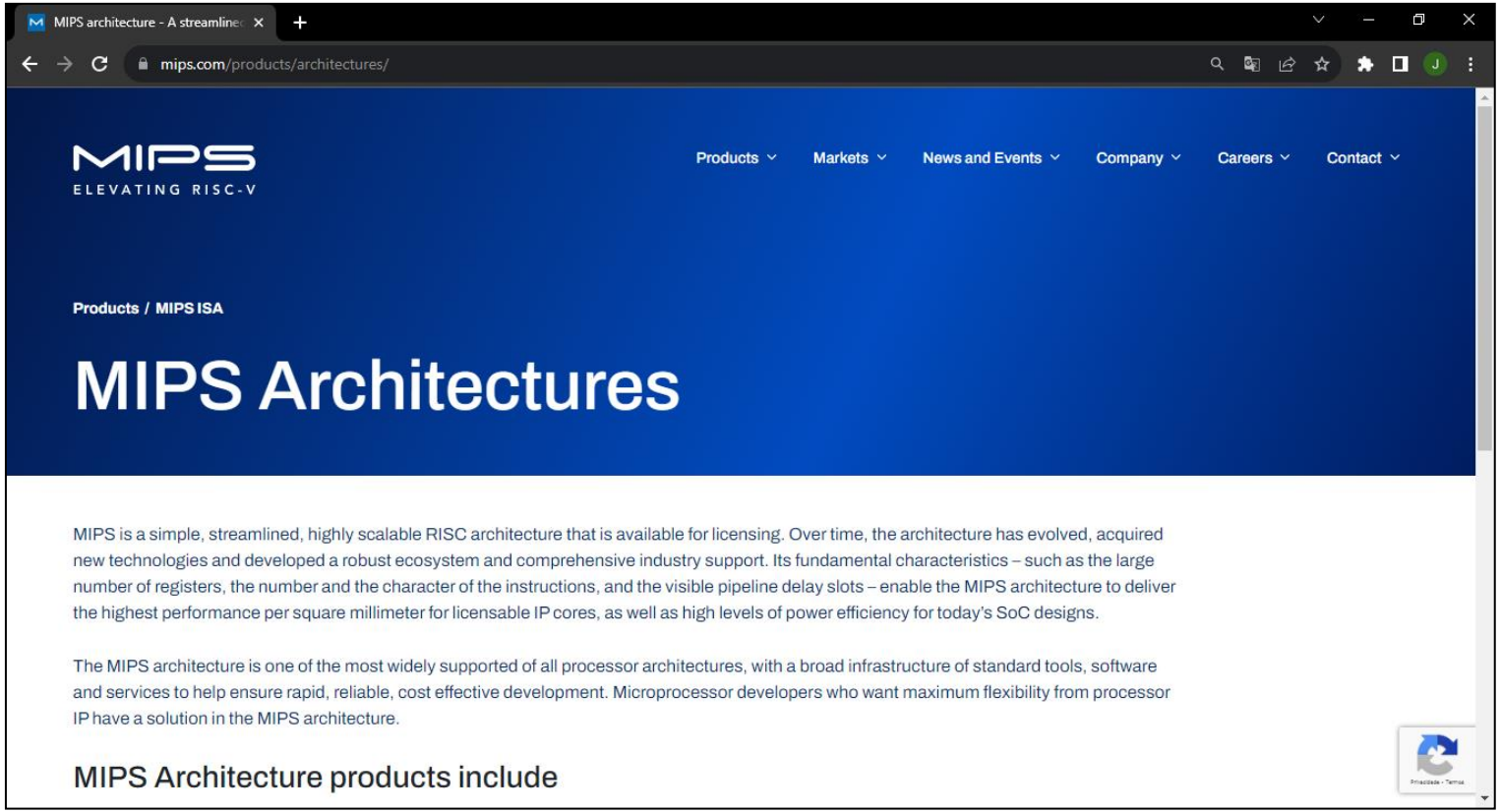
- Para controlar o hardware do computador, é necessário falar a sua linguagem.
- Palavras da linguagem do computador são chamadas de **instruções**
  - O vocabulário é chamado de **conjunto de instruções**.
- Apresentação das instruções através de uma abordagem *top-down.*
- Linguagens de computadores são semelhantes (entre diferentes arquiteturas)
  - Ao contrário da linguagem dos humanos.

Introdução

- O conjunto de instruções MIPS:
  - https://pt.wikipedia.org/wiki/Arquitetura_MIPS
  - Criado na década de 1980;
  - 100 milhões de processadores fabricados em 2002;
  - ATI, Broadcom, Cisco, NEC, Nintendo, Silicon Graphics, Sony, Texas Instruments e Toshiba.
    - Sony PlayStation (PS1): MIPS R3000A
    - Nintendo 64: NEC VR4300
    - Sony PSP: Baseado no MIPS R4000

- Processador RISC (conjunto reduzido de instruções):
  - Tamanho de instruções fixo (*fixed instruction lengths*);
  - Instruções de load-store (*load-store instruction sets*);
  - Modos de endereçamento limitado (*limited addressing modes*);
  - Operações limitadas (*limited operations*).

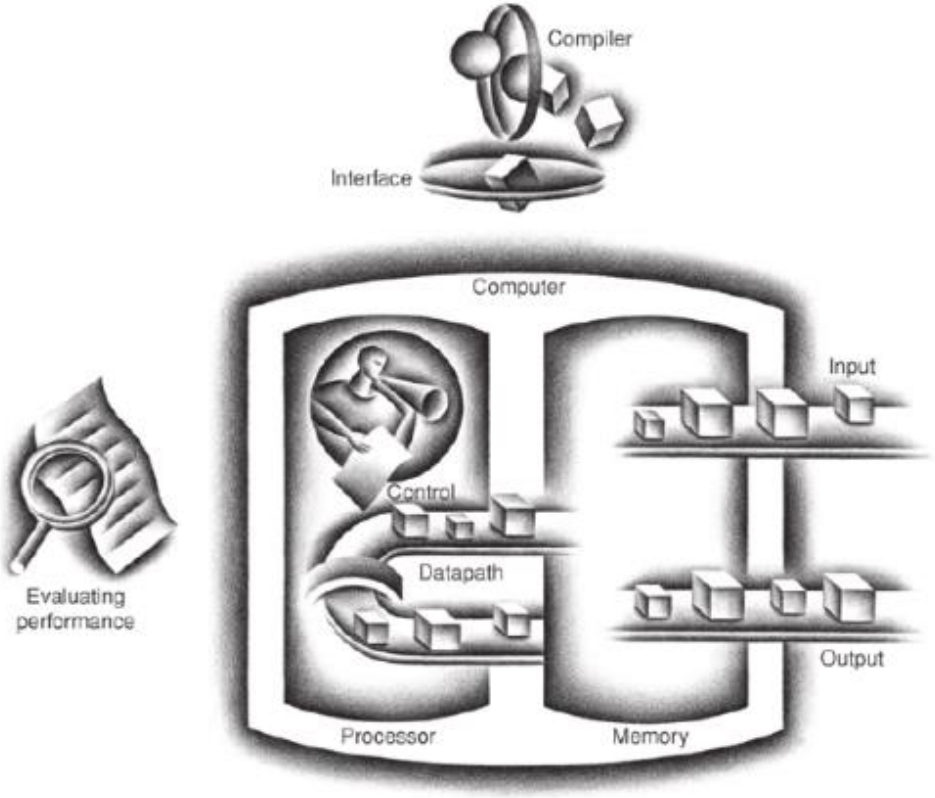
Introdução

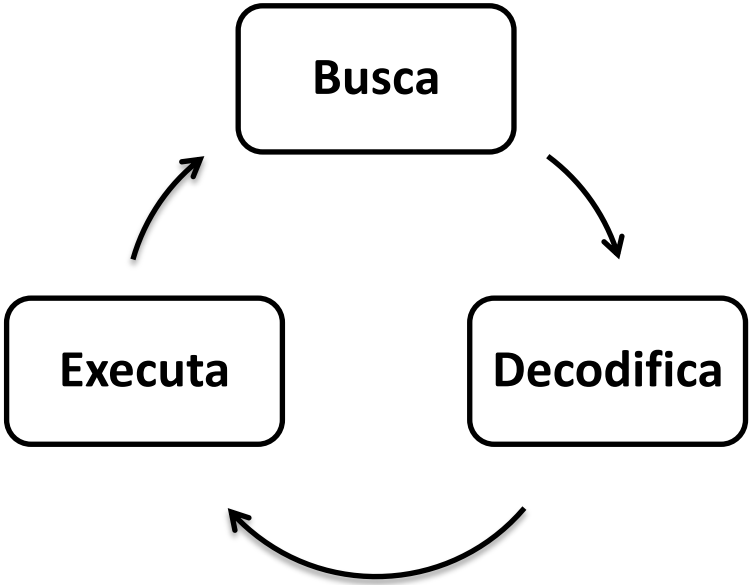


mips.com

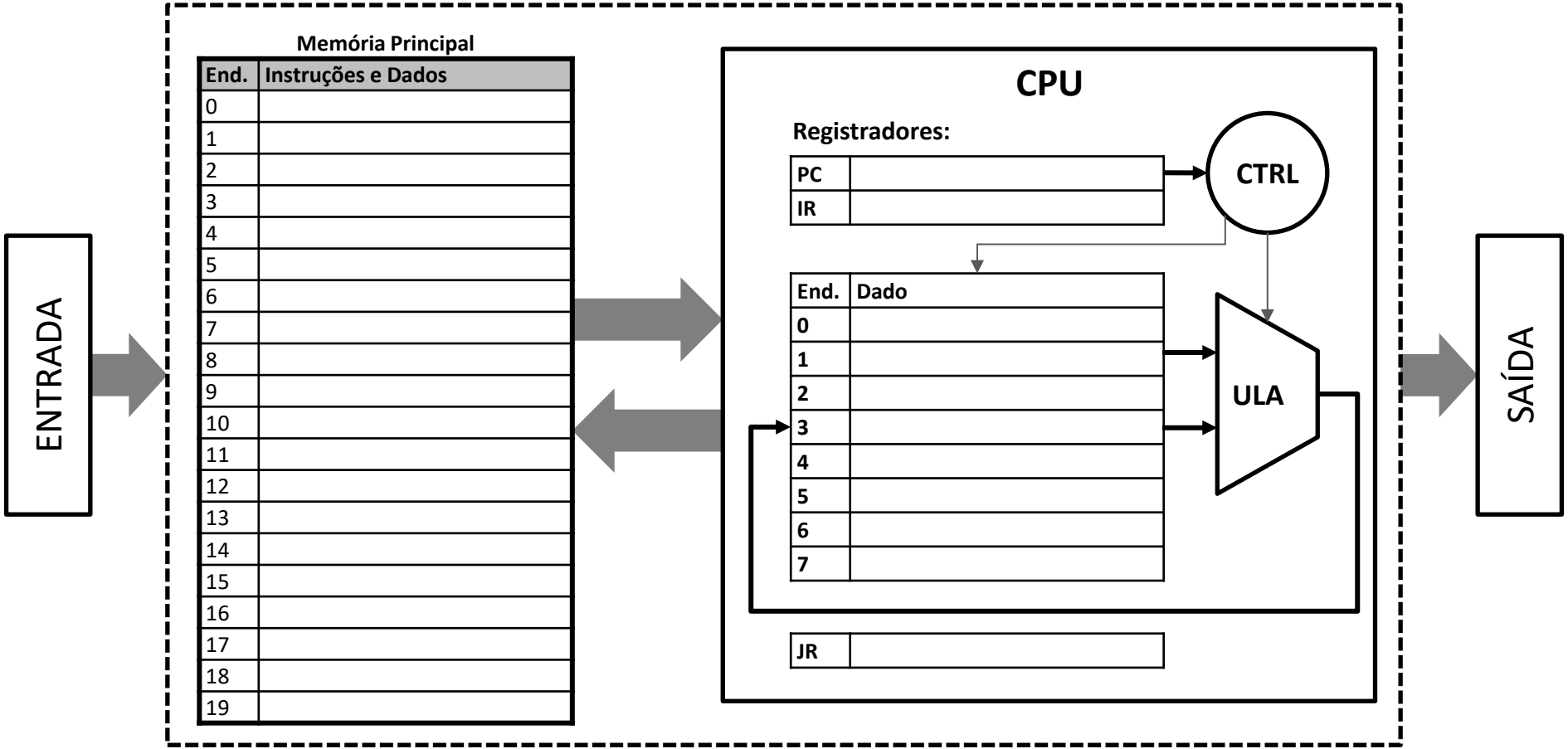
Introdução

- O ciclo básico de busca e execução de instruções.

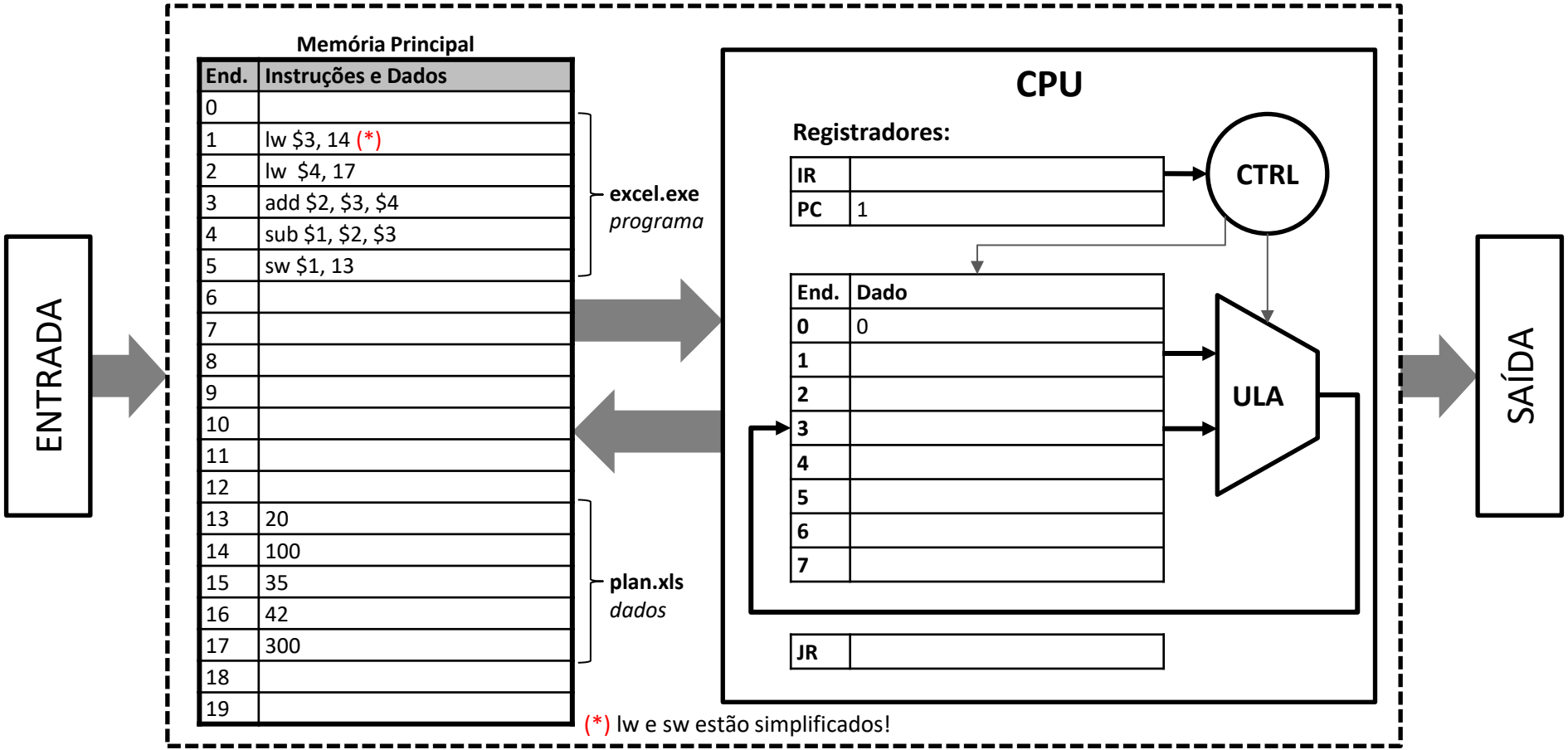




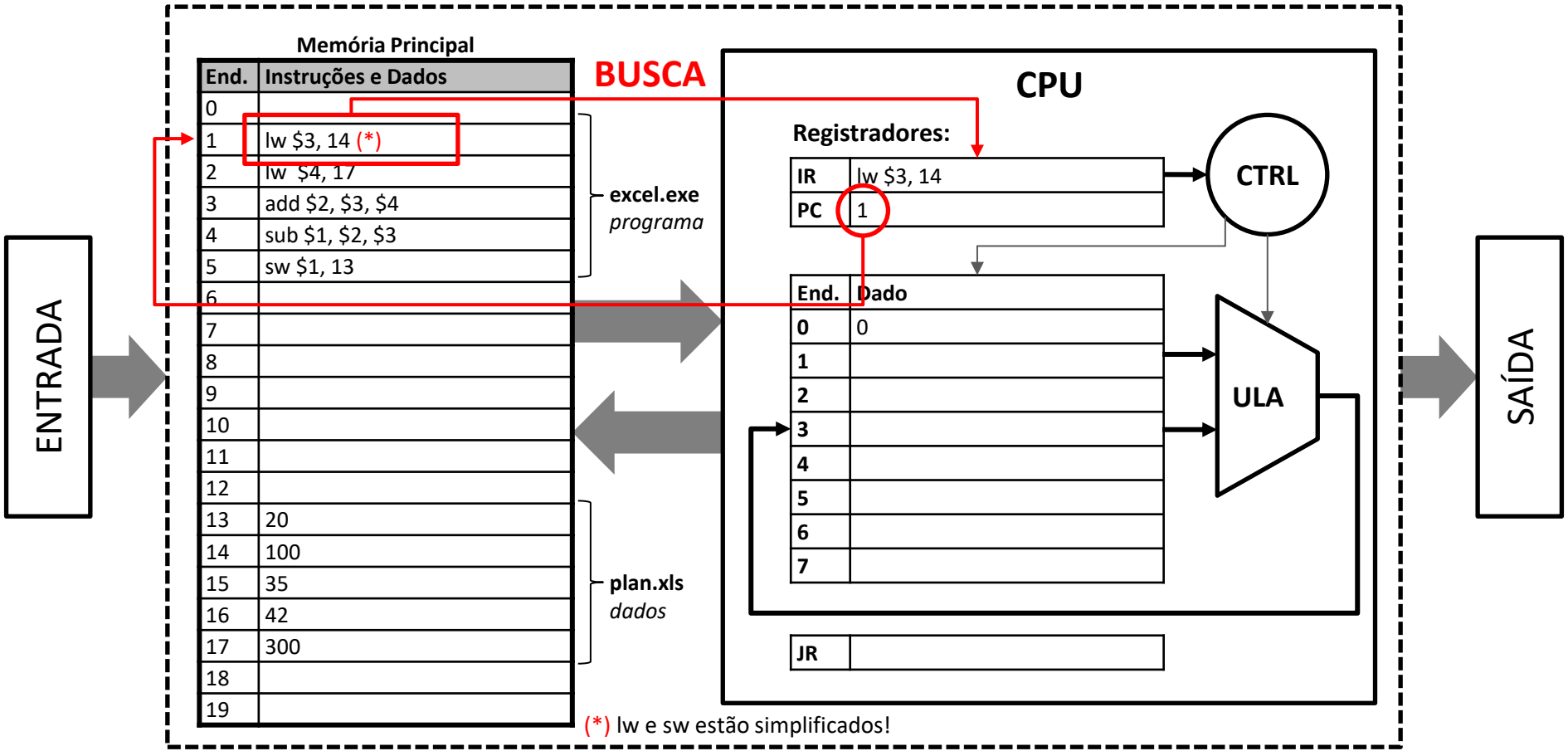
Arquitetura de Von Neumann (simplificado)



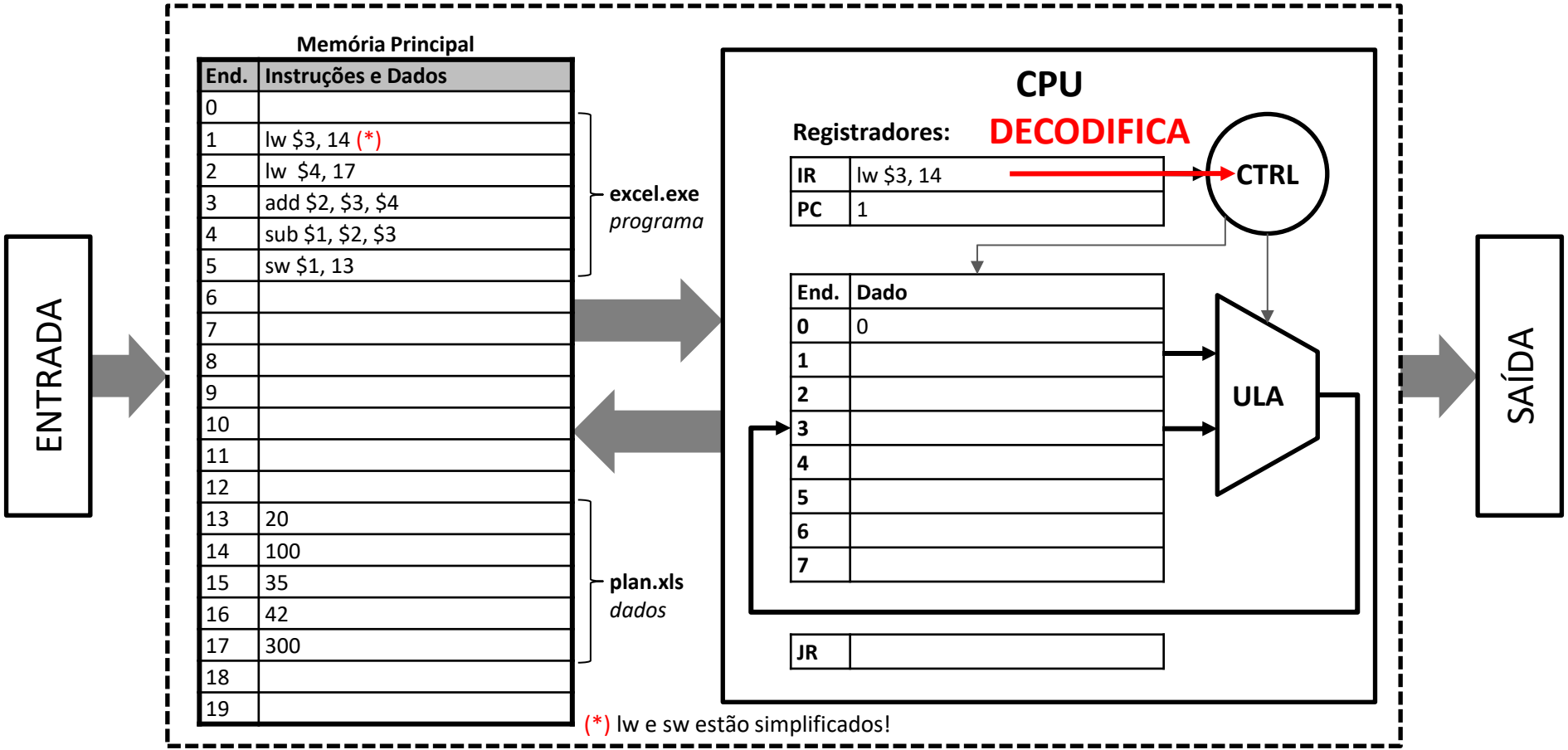
Arquitetura de Von Neumann (simplificado)



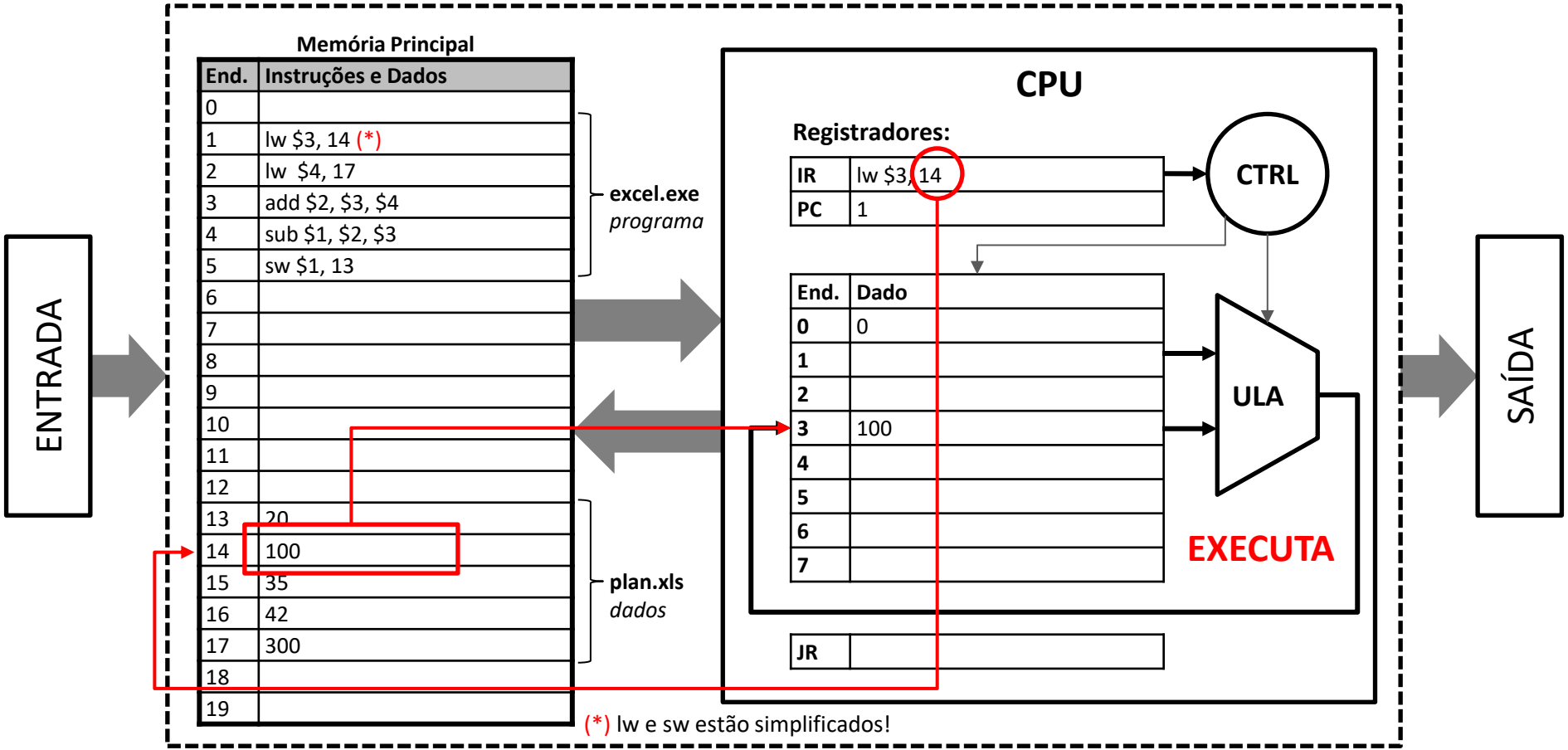
Arquitetura de Von Neumann (simplificado)



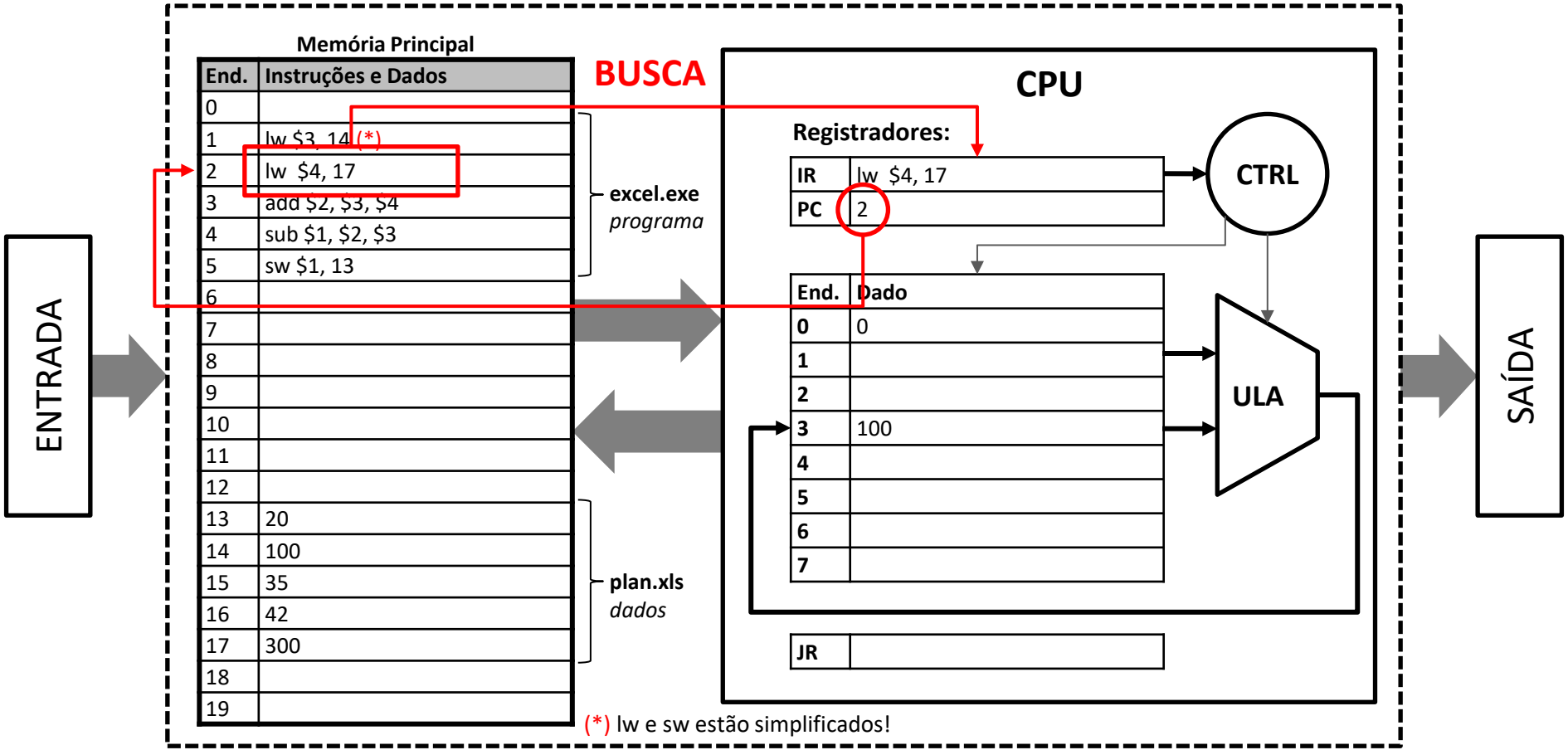
Arquitetura de Von Neumann (simplificado)



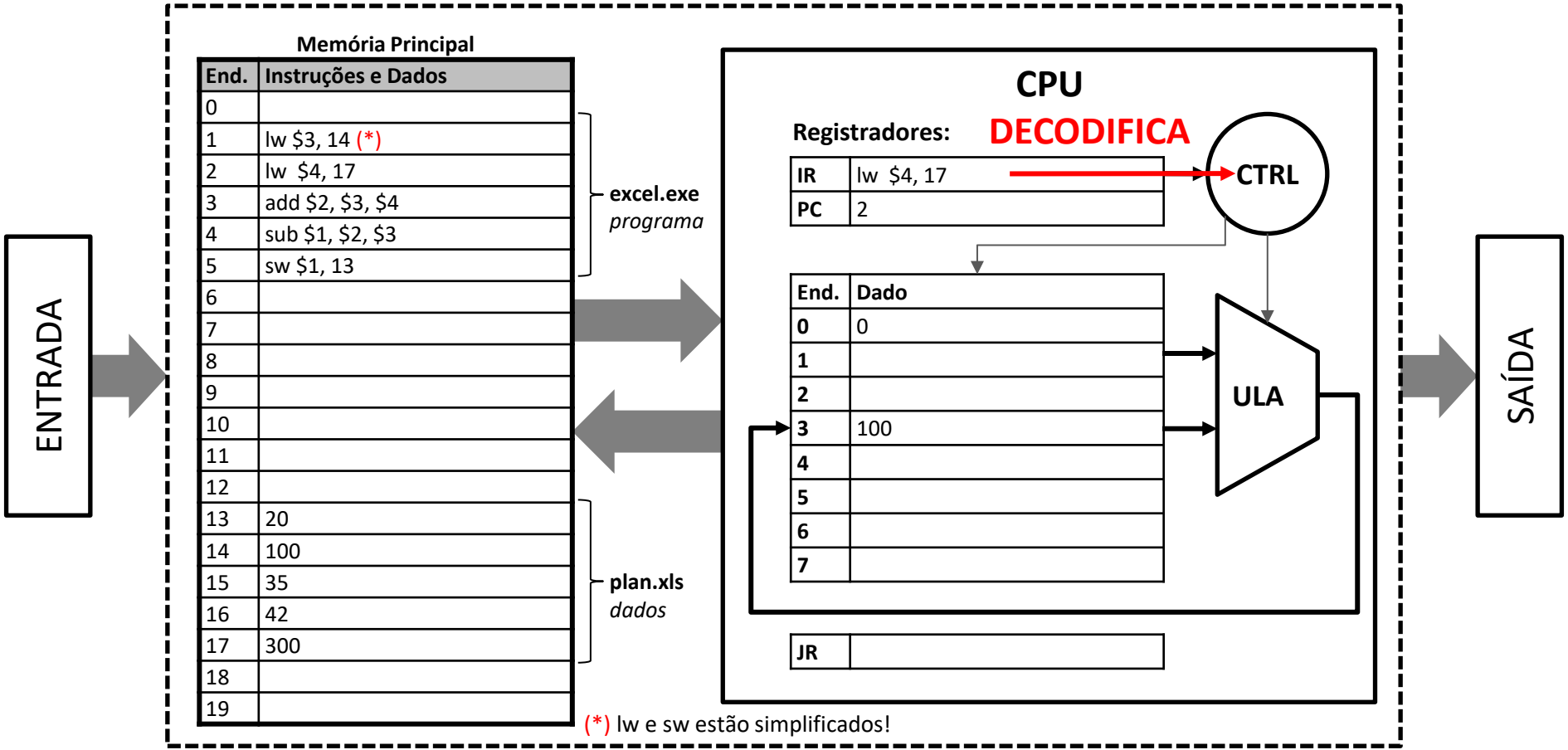
Arquitetura de Von Neumann (simplificado)



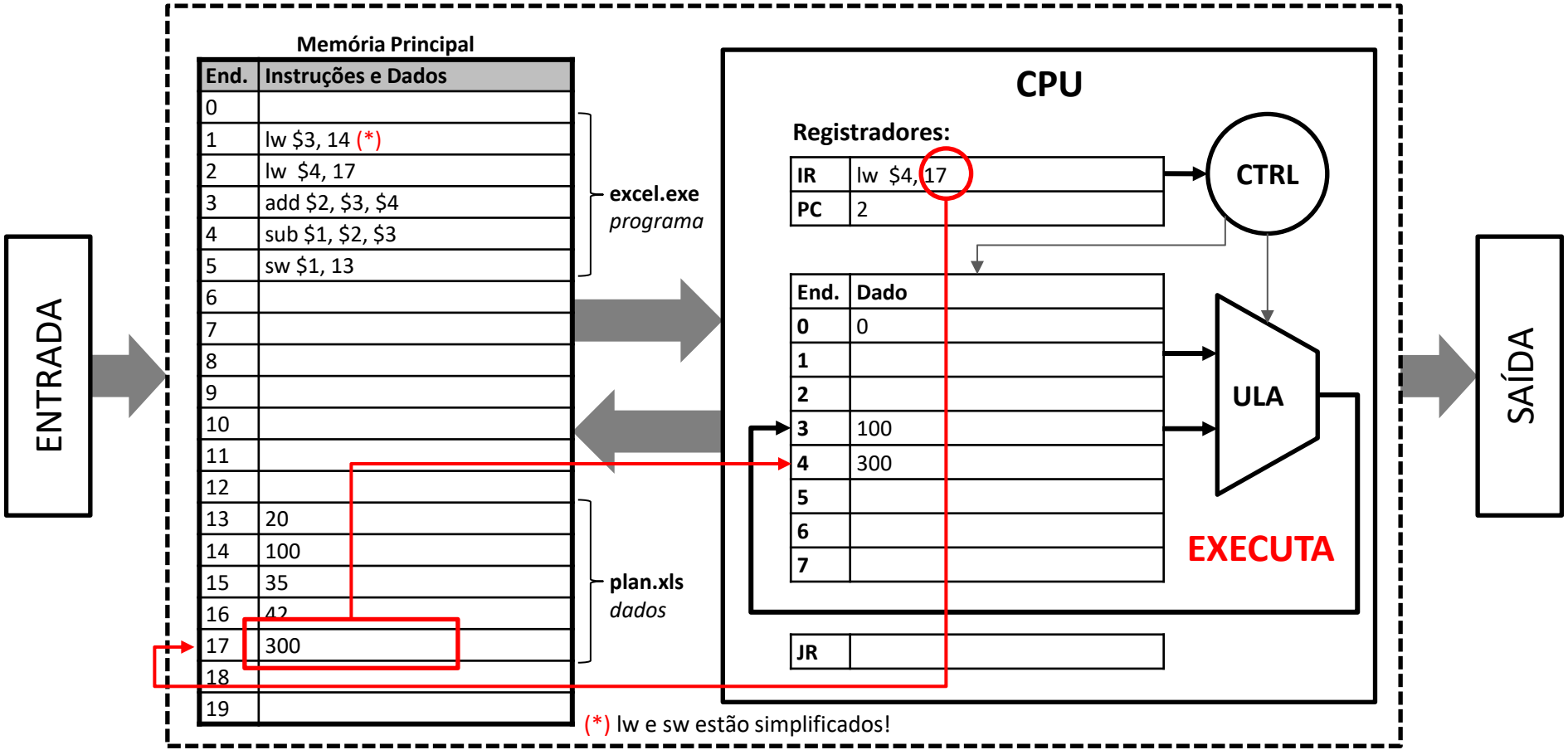
Arquitetura de Von Neumann (simplificado)



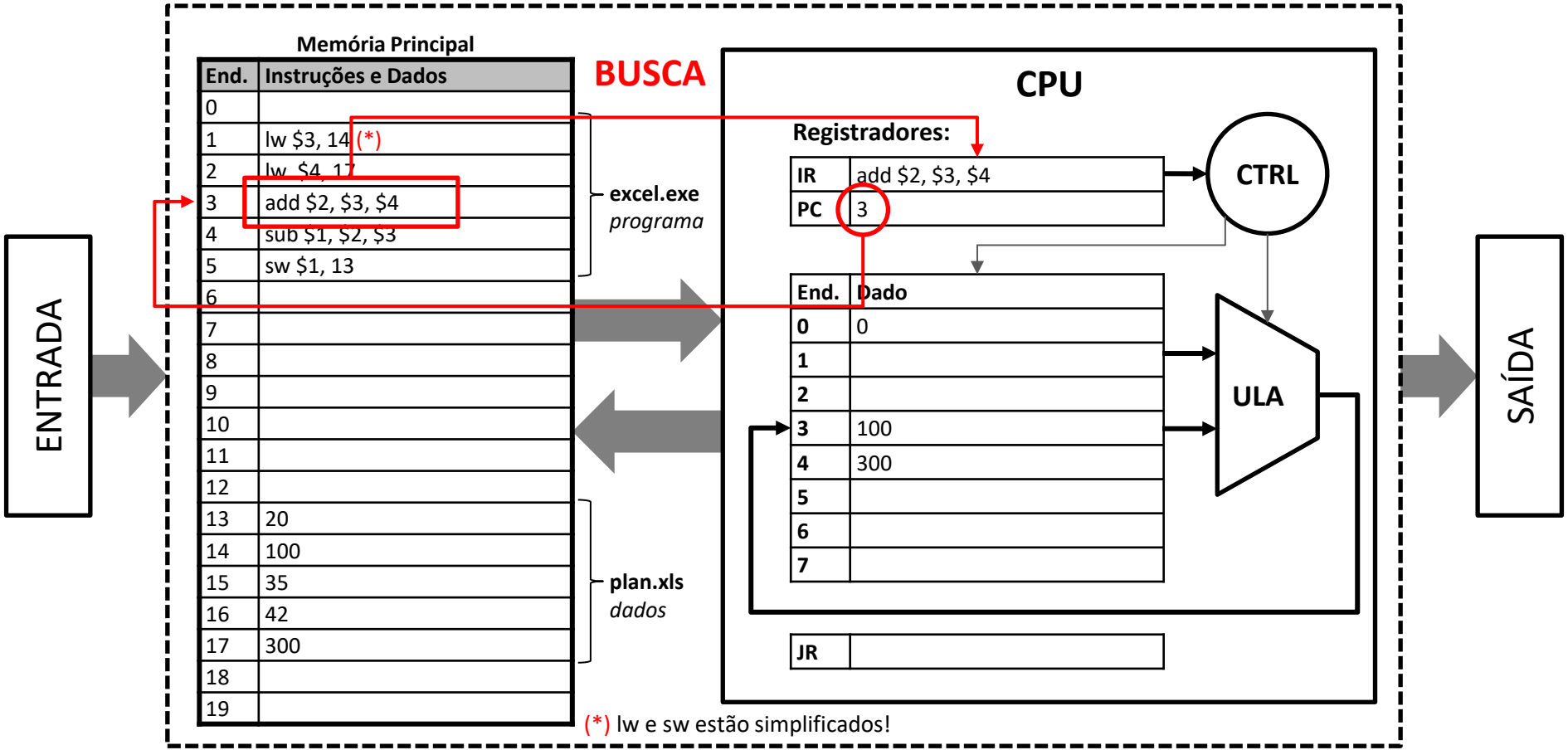
Arquitetura de Von Neumann (simplificado)



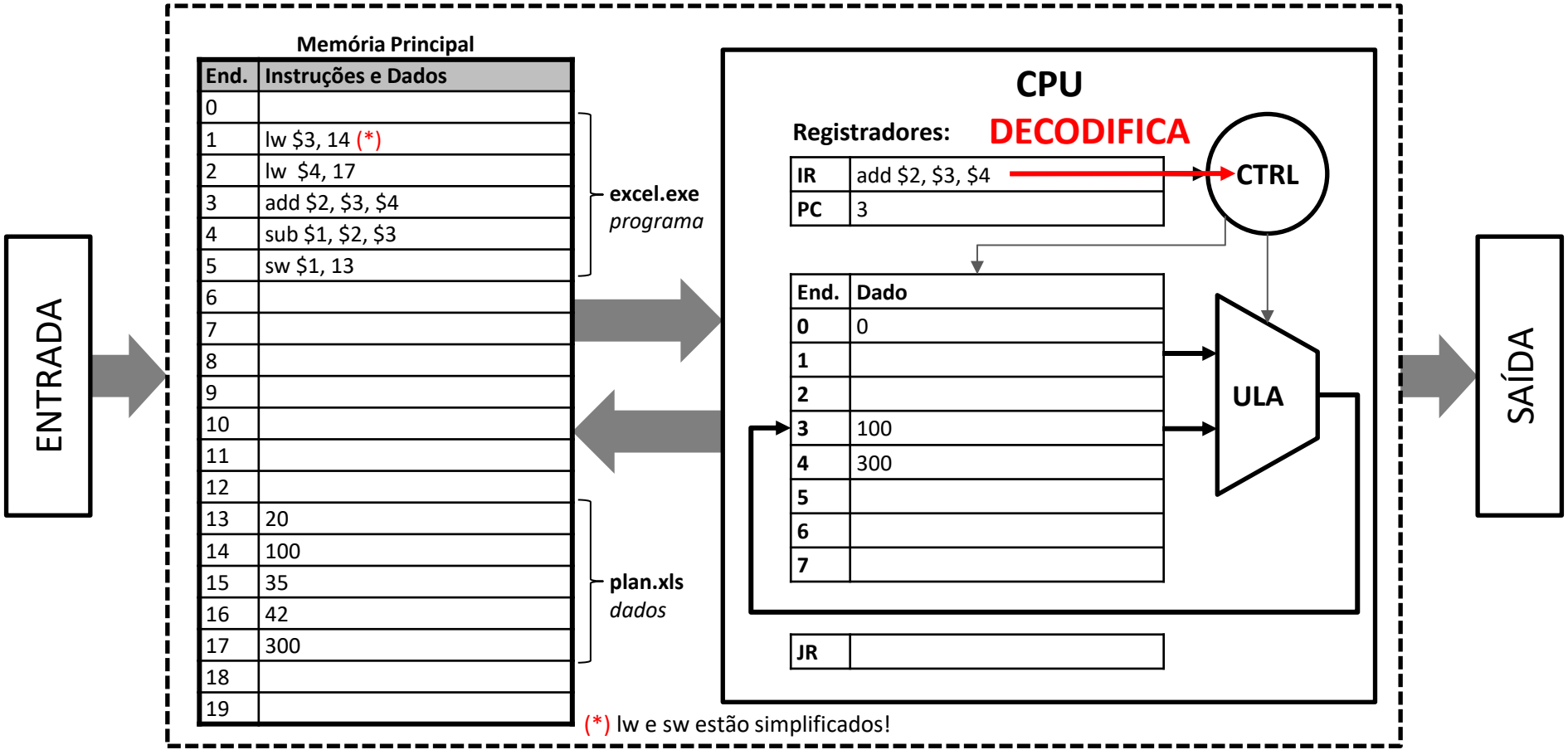
Arquitetura de Von Neumann (simplificado)



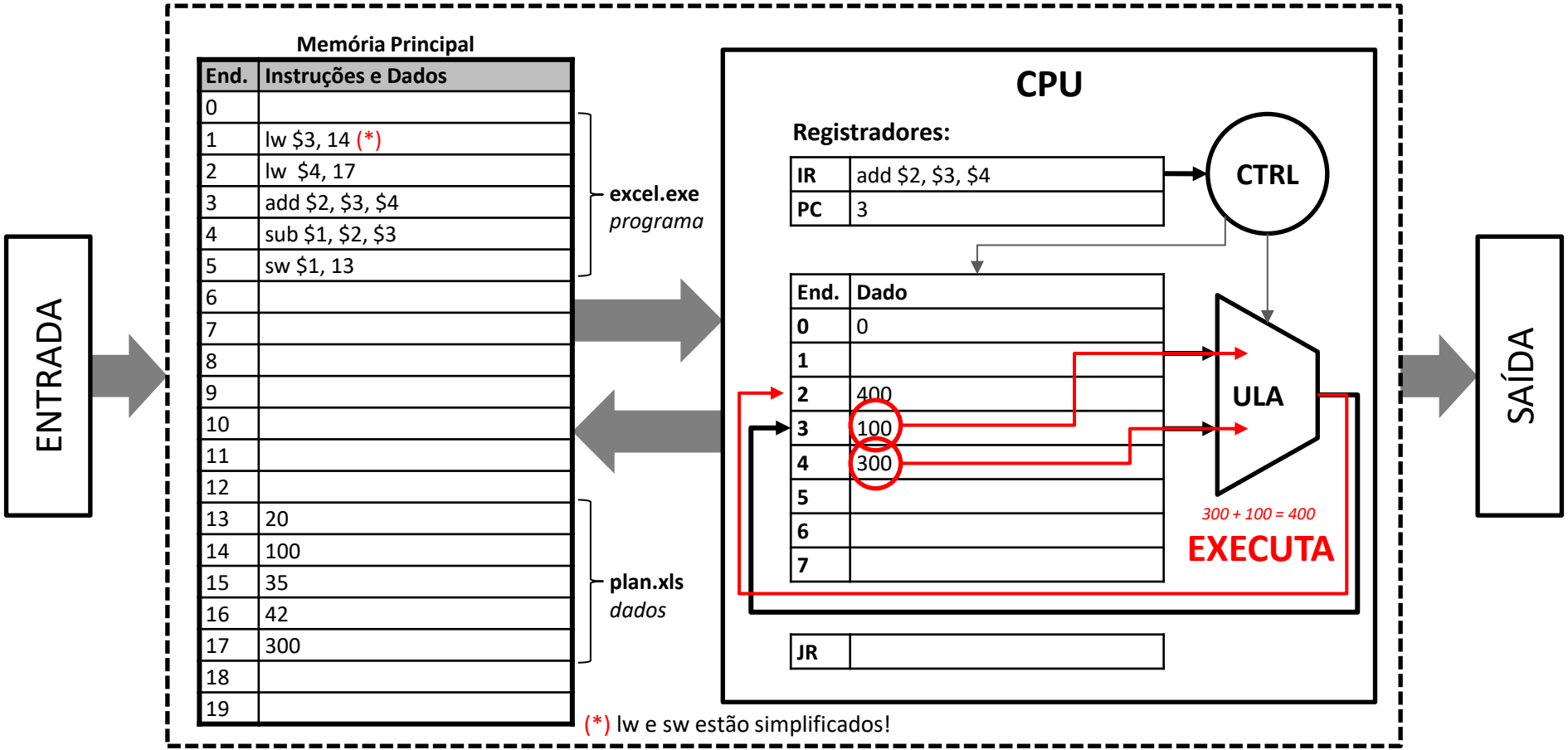
Arquitetura de Von Neumann (simplificado)



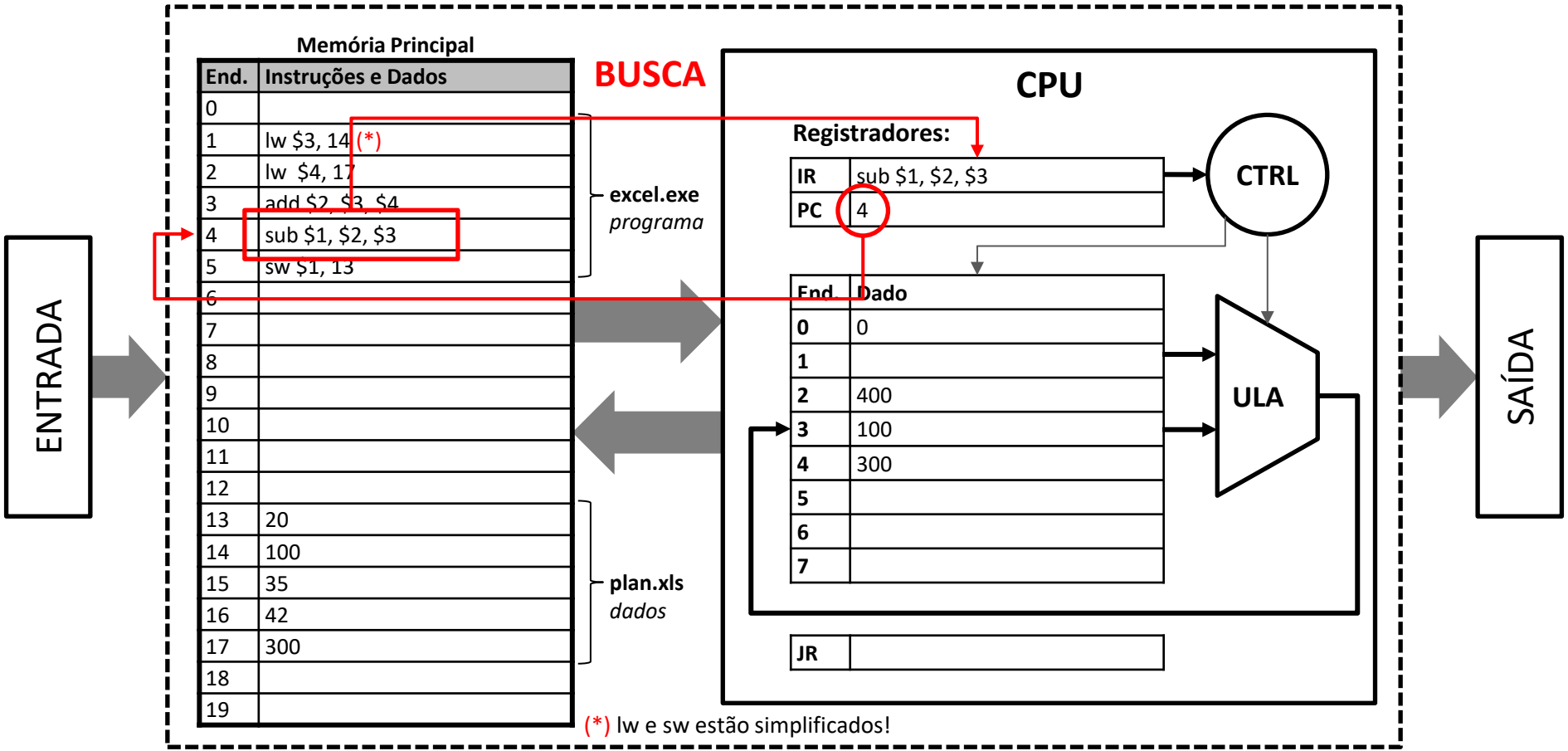
Arquitetura de Von Neumann (simplificado)



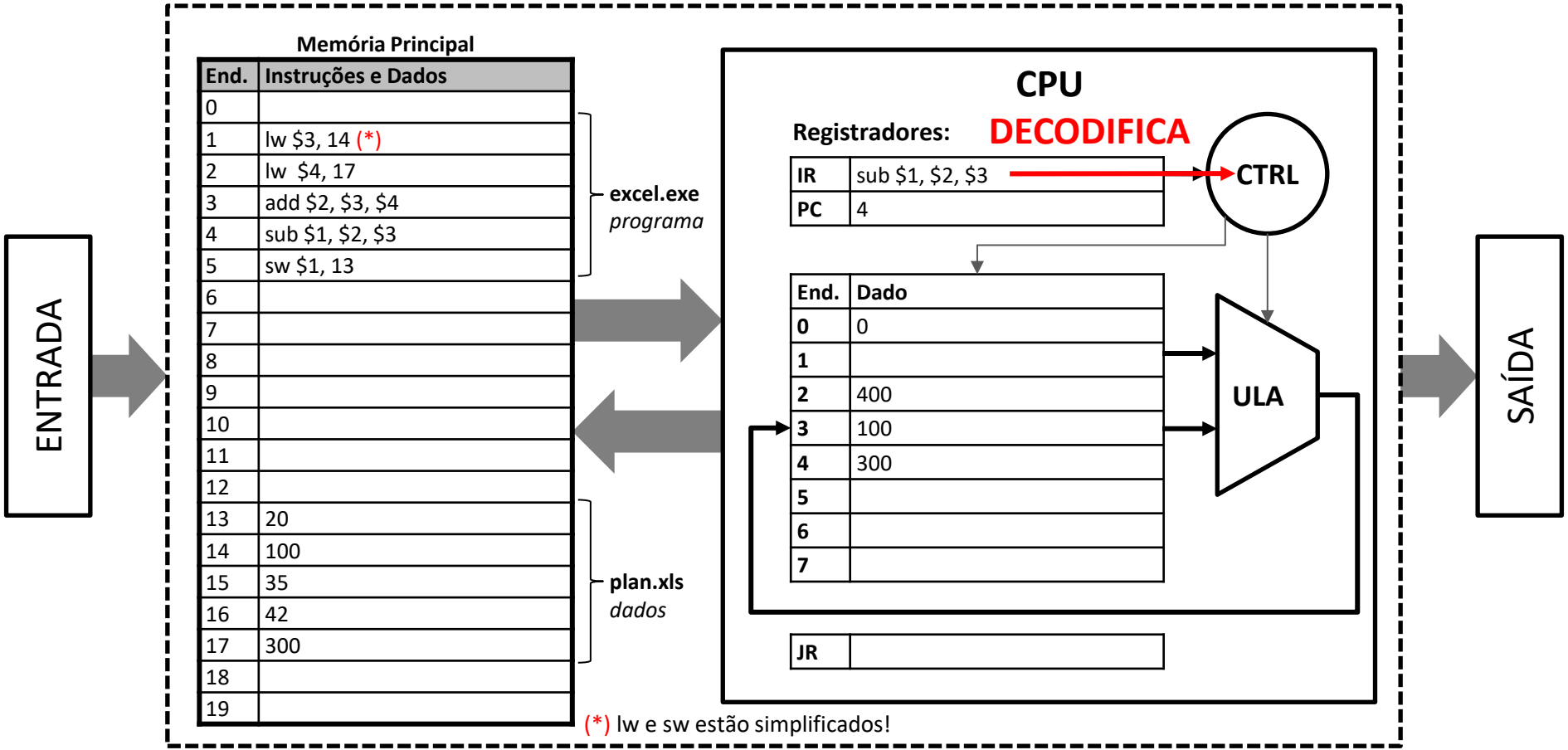
Arquitetura de Von Neumann (simplificado)



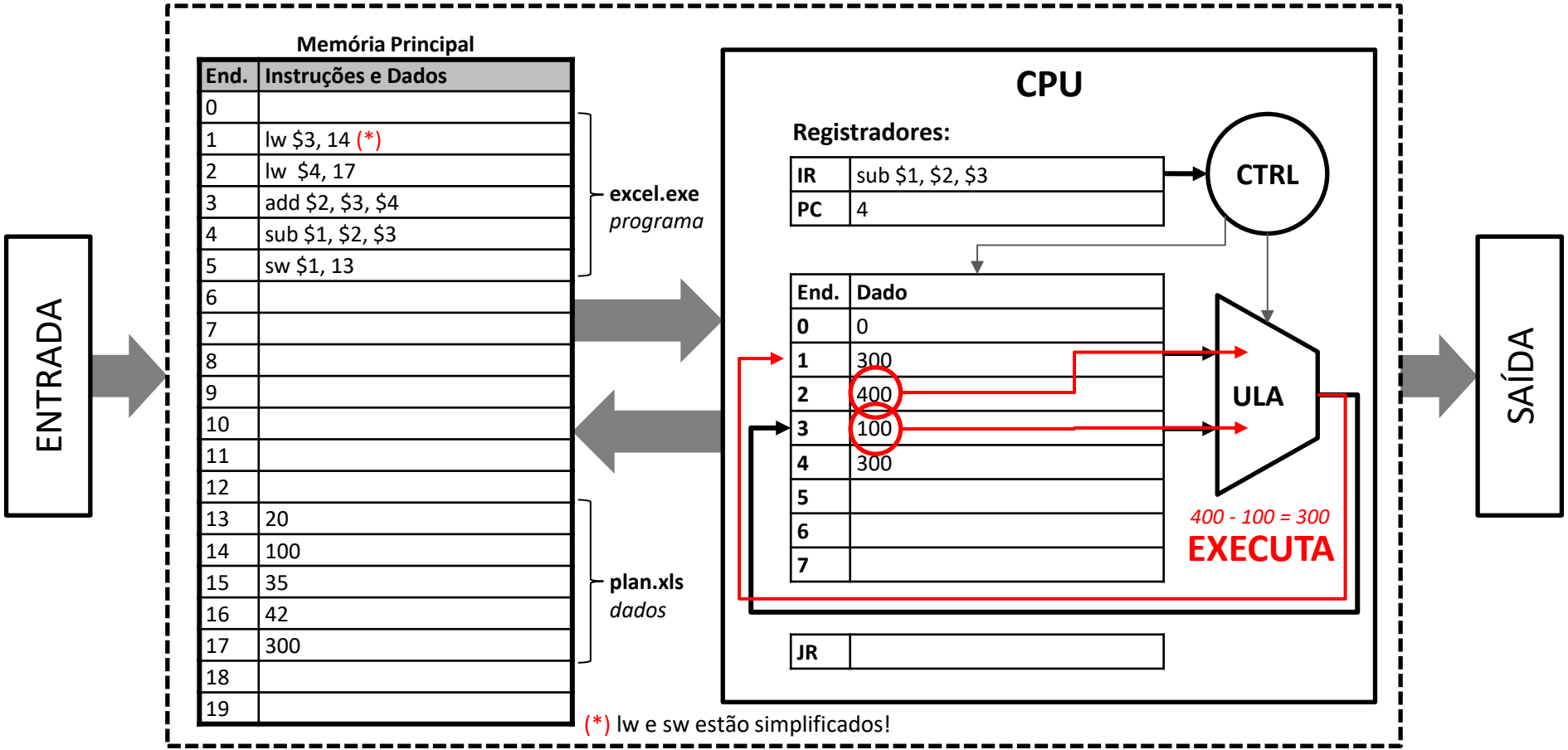
Arquitetura de Von Neumann (simplificado)



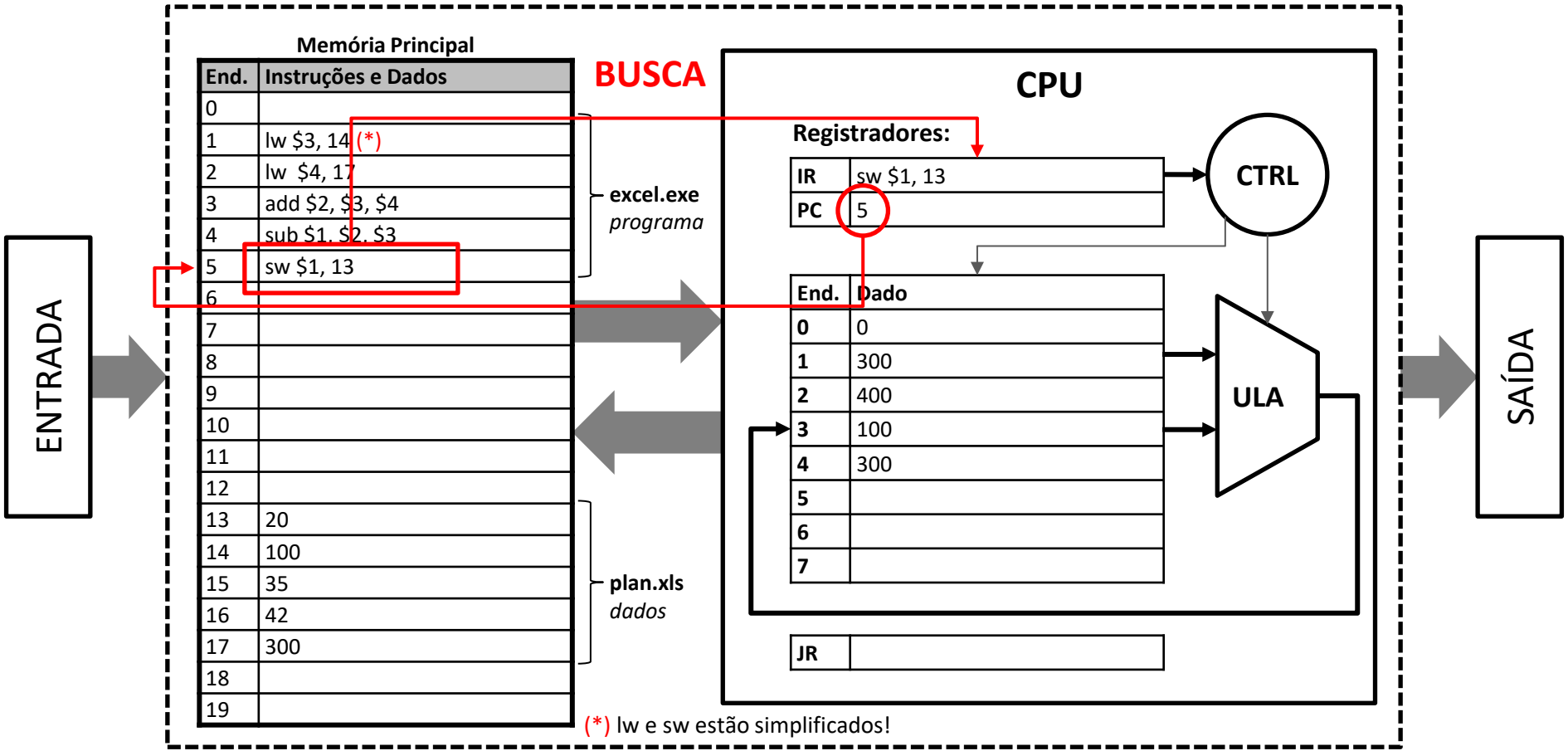
Arquitetura de Von Neumann (simplificado)



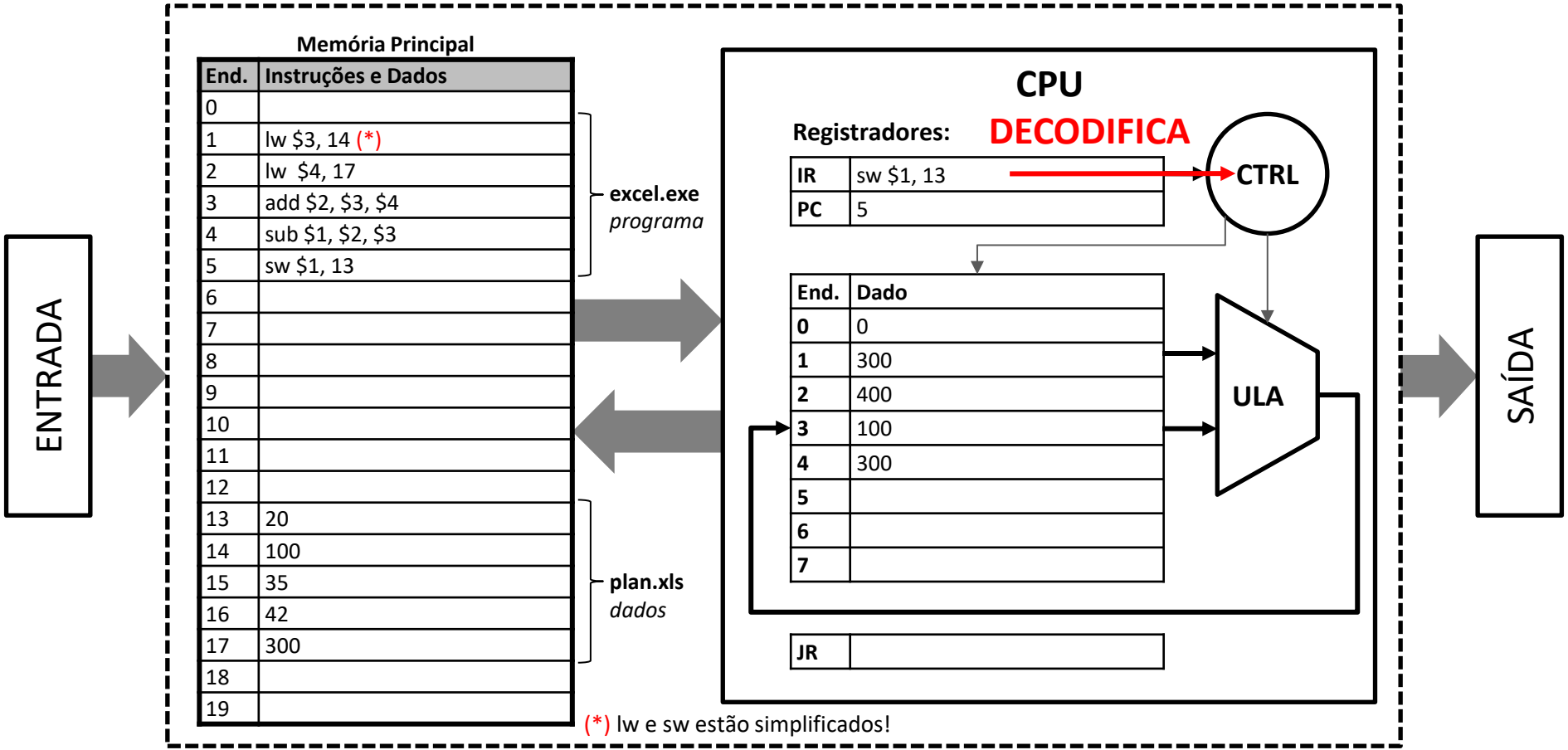
Arquitetura de Von Neumann (simplificado)



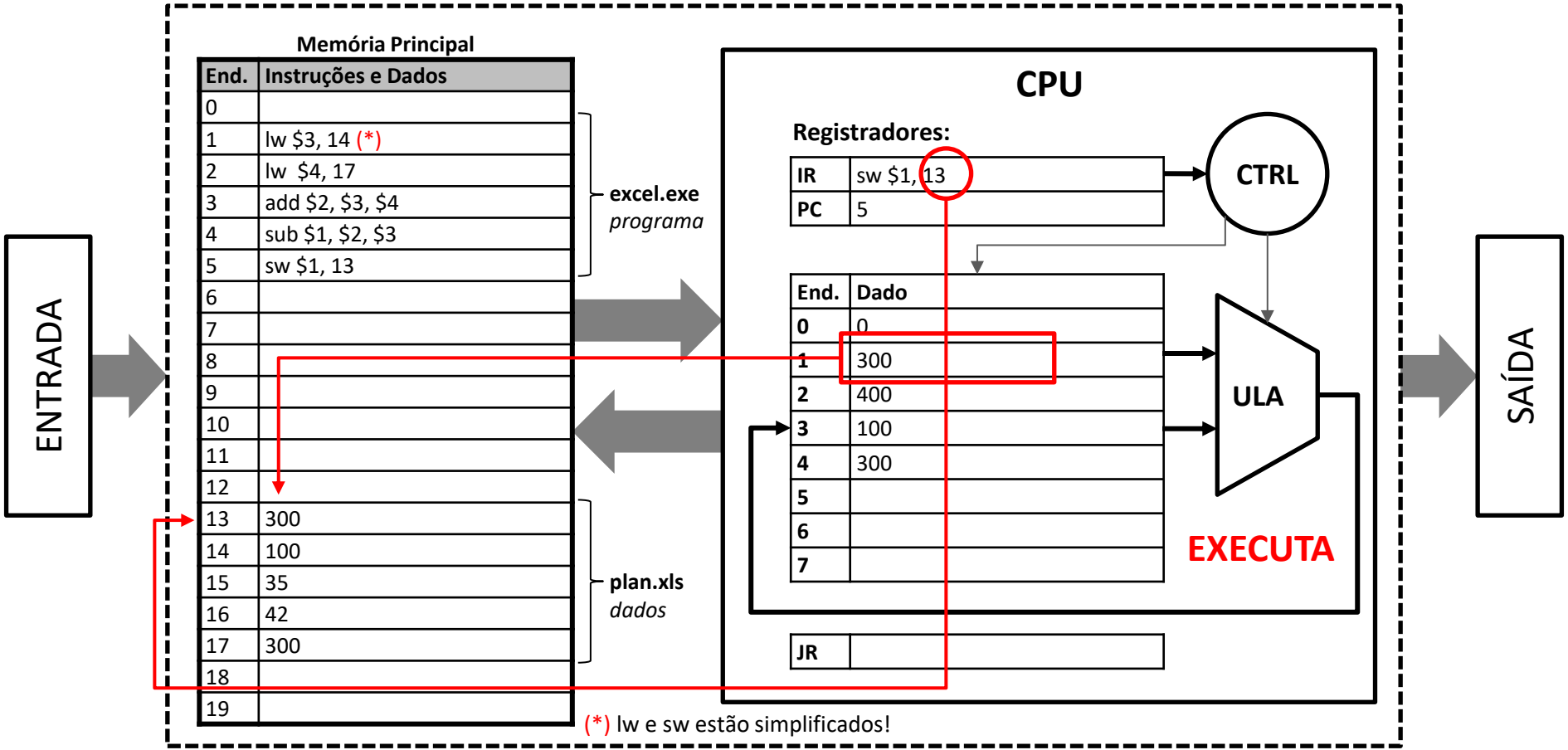
Arquitetura de Von Neumann (simplificado)



Arquitetura de Von Neumann (simplificado)



Arquitetura de Von Neumann (simplificado)



MIPS 32 – Arquitetura do Conjunto de Instruções

- Tipos de instrução:
  - Movimentação de Dados (*Load e Store*);
  - Aritmética e Lógica;
  - Saltos e Desvios (*Jump* e *Branch*);
  - Ponto flutuante;
  - Gerenciamento de memória;
  - Especiais.

FORMATOS DE INSTRUÇÃO BÁSICOS (32 bits)

	31 – 26	25 – 21	20 – 16	15 – 11	10 – 6	5 – 0
R	**opcode**	**rs**	**rt**	**rd**	**shamt**	**funct**

	31 – 26	25 – 21	20 – 16	15 – 0
I	**opcode**	**rs**	**rt**	**endereço/imediato**

	31 – 26	25 – 0
J	**opcode**	**endereço**

MIPS 32 – Arquitetura do Conjunto de Instruções

- Tipos de instrução:
  - Movimentação de Dados (*Load e Store*);
  - Aritmética e Lógica;
  - Saltos e Desvios (*Jump* e *Branch*);
  - Ponto flutuante;
  - Gerenciamento de memória;
  - Especiais.

FORMATOS DE INSTRUÇÃO BÁSICOS (32 bits)

	31 – 26	25 – 21	20 – 16	15 – 11	10 – 6	5 – 0
R	**opcode**	**rs**	**rt**	**rd**	**shamt**	**funct**

	31 – 26	25 – 21	20 – 16	15 – 0
I	**opcode**	**rs**	**rt**	**endereço/imediato**

	31 – 26	25 – 0
J	**opcode**	**endereço**

OPERAÇÕES NO HARDWARE DO COMPUTADOR

Operações no hardware do computador

- Todo computador precisa realizar operações aritméticas:
  - **Ex:** `add a, b, c`
    - Instrui o computador para realizar a soma entre as variáveis `b` e `c` e armazenar o resultado em `a`.
- **Ex:** Colocar a soma de `b`, `c`, `d`, e `e` na variável `a`:
  - Em linguagem de alto nível

```
1. a = b + c + d + e;
```

  - Em Assembly MIPS:

```
1. add a, b, c    # soma b + c é colocada em a
2. add a, a, d    # soma b + c + d está em a
3. add a, a, e    # soma b + c + d + e está em a
```

- Comentário no assembly do MIPS: `#`

Operações no hardware do computador

- O número natural de operandos para uma operação de adição é três
- Dessa forma, cada instrução deve tenha exatamente três operandos
  - Mantem o hardware simples
  - Hardware para um número variável de operandos é mais complicado do que para um número fixo

- **Princípio de projeto 1:**
  - Simplicidade favorece a regularidade.

Operações no hardware do computador

- Compilando duas instruções de atribuição C no MIPS:

```
1. a = b + c;
2. d = a - e;
```

- A tradução é realizada pelo compilador (MIPS):

```
1. add a, b, c
2. sub d, a, e
```

Ex: Compilando uma instrução complexa no MIPS

- Compilando uma instrução complexa no MIPS

```
1. f = (g + h) - (i + j);
```

- O compilador precisa desmembrar essa instrução em várias instruções *assembly*
  - Somente uma operação é realizada por instrução MIPS.

```
1. add t0, g, h      # var. temp. t0 contém g+h
2. add t1, i, j      # var. temp. t1 contém i+j
3. sub f, t0, t1     # f recebe t0 - t1
```

  - Uma única expressão em linguagem C gera três instruções *assembly* para o MIPS.

Ex: Compilando uma instrução complexa no MIPS

- Compilando uma instrução complexa no MIPS

```
1. f = (g + h) - (i + j);
```

- O compilador precisa desmembrar essa instrução em várias instruções *assembly*
  - Somente uma operação é realizada por instrução MIPS.

```
1. add t0, g, h      # var. temp. t0 contém g+h
2. add t1, i, j      # var. temp. t1 contém i+j
3. sub f, t0, t1     # f recebe t0 - t1
```

  - Uma única expressão em linguagem C gera três instruções *assembly* para o MIPS.

- Neste exemplo, usamos os nomes das variáveis por simplificação.
  - No MIPS, as operações utilizam registradores como operandos.
  - Veremos isso na próxima seção.

OPERANDOS NO HARDWARE DO COMPUTADOR

Operandos no hardware do computador

- Os operandos das instruções precisam estar armazenados nos registradores
  - **Registradores:**
    - Grupo limitado de locais de armazenamento
    - Embutidos diretamente no hardware (CPU)
    - Visíveis para o programador

- O MIPS possui 32 registradores:
  - Cada registrador possui 32 bits.
  - Grupos de 32 bits ocorrem com tanta frequência no MIPS que recebem o nome de palavra (*word*).
  - Grupos específicos de registradores possuem funções definidas.
    - `$s0, $s1, ..., $s7`
      - Correspondem as variáveis dos programas C e Java.
    - `$t0, $t1, ..., $t7`
      - Registradores temporários necessários para compilar o programa.

Operandos no hardware do computador

Nome	Número do registrador	Uso	Valor preservado
$zero	0	Constante 0	n.a.
$at	1	Reservado para o montador	n.a.
$v0 - $v1	2-3	Valores retornados	não
$a0 - $a3	4-7	Argumentos	sim
$t0 - $t7	8-15	Temporários	não
$s0 - $s7	16-23	Valores salvos	sim
$t8 - $t9	24-25	Temporários	não
$k0 - $k1	26-27	Temporários SO	sim
$gp	28	Ponteiro global	sim
$sp	29	Ponteiro para pilha	sim
$fp	30	Ponteiro para quadro	sim
$ra	31	Endereço de retorno	sim

Operandos no hardware do computador

- Variáveis em programas em linguagem de alto nível são ilimitadas
- O número de registradores é limitado
- O MIPS possui 32 registradores:
  - Quantidade muito grande de registradores:
    - Aumentar o tempo do ciclo de *clock:*
      - Os sinais eletrônicos levam mais tempo para atravessar distâncias maiores
    - Necessita de um número maior de bits para representação:
      - Influencia o tamanho da instrução
  - Deve-se equilibrar o “desejo” dos programas por mais registradores com o desejo do projetista de manter o ciclo de *clock* mais rápido.

- **Princípio de projeto 2:**
  - Menor significa mais rápido

Ex: Compilando uma atribuição em C usando registradores

- Compilando uma atribuição em C usando registradores:

```
1. f = (g + h) - (i + j);
```

  - As variáveis `f`, `g`, `h`, `i` e `j` estão associadas aos registradores `$s0`, `$s1`, `$s2`, `$s3` e `$s4` respectivamente

- Código MIPS compilado:

```
1. add $t0, $s1, $s2     # reg. $t0 contém g + h
2. add $t1, $s3, $s4     # reg. $t1 contém i + j
3. sub $s0, $t0, $t1     # reg. $s0 contém $t0 - $t1,
4.                       #      que é (g + h) - (i + j)
```

a02.ex01.asm

Operandos no hardware do computador

- **Operandos na memória:**
  - Operações aritméticas só ocorrem com os registradores nas instruções MIPS:
  - O MIPS deve ter instruções que transferem os dados entre o processador e a memória:
    - Instruções de transferência de dados.
  - Para acessar uma palavra (*word*) na memória, é necessário passar o **endereço de memória** a ser utilizado.

Operandos no hardware do computador

- A instrução *load word* (código mnemônico, `lw`):
  - Copia dados da memória para um registrador
  - Sintaxe:

```
1. lw registrador-destino, offset(registrador-base)
```

  - Transfere o conteúdo na posição de memória registrado-base + offset (*) para o registrador-destino
  - Ex:

```
1. lw $t0, 8($s3)  # $t0 é o registrador destino
2.                 # 8 é o offset, uma constante de
3.                 #     deslocamento.
4.                 # $s3 é o registrador base
```

- (*) CUIDADO! Ainda não estamos considerando a restrição de alinhamento.

Ex: Compilando quando um operador está na memória

- Considere `A` é uma sequência de 100 palavras (arranjo)
  - O compilador associou as variáveis `g` e `h` aos registradores `$s1` e `$s2`.
  - O endereço inicial da sequencia esta armazenado em `$s3` (endereço base)

```
1. g = h + A[8]; // código em linguagem C
```

- Embora haja uma única operação nessa instrução, um dos operandos está na memória
  - Precisamos transferir `A[8]` para um registrador temporário:

```
1. lw $t0, 8($s3) # registrador temporário $t0 recebe A[8]
```

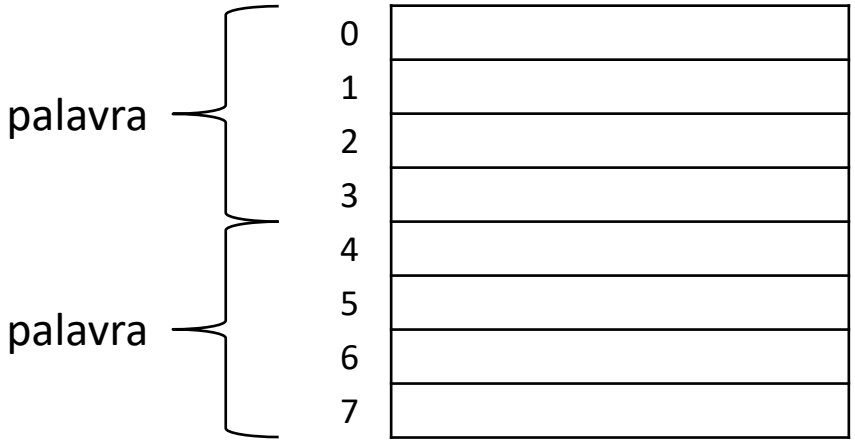
    - `$s3` contém o endereço de base – o endereço de `A[0]`.
    - `8` é o *offset* e define uma constante de deslocamento **(*)**
    - Registrador somado ao *offset* → registrador de base
  - A instrução seguinte (`add`) pode operar sobre o valor em `$t0`, já que é um registrador:

```
2. add $s1, $s2, $t0
```

- **(*)** CUIDADO! Ainda não estamos considerando a restrição de alinhamento.

Operandos no hardware do computador

- Além de associar variáveis a registradores, o compilador aloca estrutura de dados, como vetores, em locais na memória:
  - Bytes, grupos de 8 bits, são úteis em muitos programas:
    - A maioria das arquiteturas endereça bytes individuais.
  - Endereços de palavras combinam os endereços dos 4 bytes em uma palavra (32 bits)
- No MIPS, palavras precisam começar em endereços que sejam múltiplos de 4:
  - **Restrição de Alinhamento**



Operandos no hardware do computador

- Restrição de alinhamento no MIPS:
  - No MIPS, palavras precisam começar em endereços que sejam múltiplos de 4
- O endereçamento em bytes afeta o índice do *array*:
  - Para obter o endereço em bytes a quantidade de palavras é multiplicada por 4
    - 8 palavras. Offset = 8 × 4 = 32

- A instrução complementar ao *load word* é o *store word* (mnemonico sw)*:*
  - Copia dados de um registrador para a memória;
  - Sintaxe:

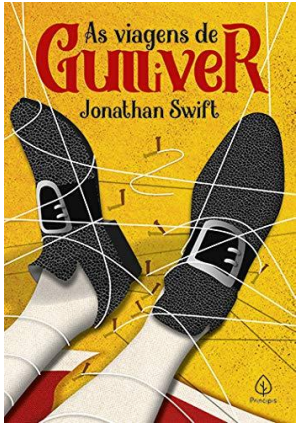
```
1. sw registrador-origem, offset(registrador-base)
```

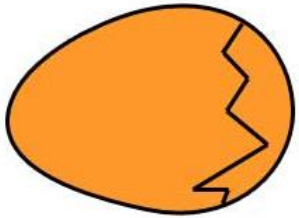
  - Ex:

```
1. sw $t0, 16($s3)  # $t0 é o registrador origem
2.                  # 16 é o offset. 4 palavras de 32 bits.
3.                  # $s3 é o registrador base
```

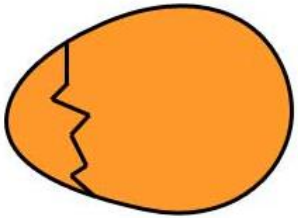
Endianness (Extremidade ou ordenação)

- *Big-endian* Vs. *Little-endian*
  - *Big-endian:* o **byte mais significativo** é armazenado no menor endereço.
  - *Little-endian:* o **byte menos significativo** é armazenado no menor endereço.
- O nome é inspirado na história “As viagens de Gulliver”
- Processadores DEC e IBM são *little-endian*. Motorola e SUN são *big-endian*.
- MIPS podem ser configurados para funcionar como *big-endian* ou *little-endian*.
  - https://pt.wikipedia.org/wiki/Extremidade_(ordenação)

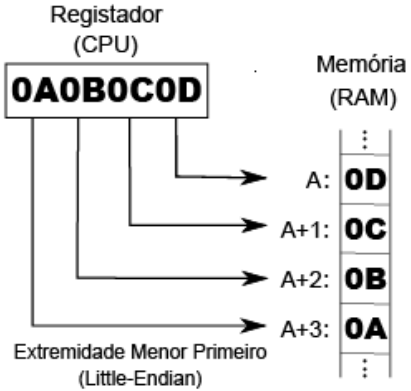


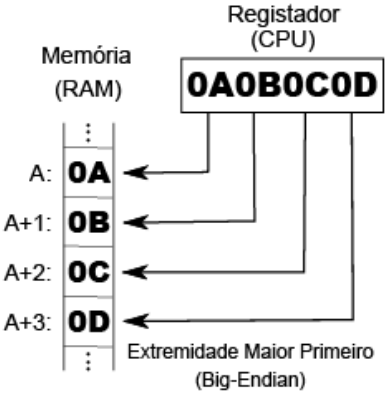


BIG ENDIAN - The way people always broke their eggs in the Lilliput land



LITTLE ENDIAN - The way the king then ordered the people to break their eggs





Operandos no hardware do computador

- Suponha que a variável `h` esteja associada ao registrador `$s2` e o endereço base do vetor `A` esteja armazenado em `$s3`.
  - Qual código assembly do MIPS para a instrução de atribuição C a seguir?

```
1. A[12] = h + A[8];
```

- Embora seja uma única instrução na linguagem C, dois operandos estão na memória;
  - São necessárias instruções para acessar os dois operandos da memória:

```
1. lw $t0, 32($s3)     # reg. temp. $t0 recebe A[8]
2. add $t0, $s2, $t0   # reg. temp. $t0 recebe h + A[8]
3. sw $t0, 48($s3)     # armazena o resultado em A[12]
```

a02.ex02.asm

Operandos no hardware do computador

- Interface hardware/software:
  - Geralmente, existem mais variáveis nos programas do que registradores nos computadores.
  - O compilador tenta manter as variáveis mais utilizadas nos registradores e coloca as restantes na memória
    - Usando *load* e *store* para movimentar os dados
  - *Spilling registers:*
    - É o processo de colocar variáveis menos utilizadas, ou aquelas que só serão empregadas mais tarde, na memória.
  - Registradores, apesar de serem mais reduzidos e terem um tamanho menor que a memória principal, são mais rápidos:
    - Preocupação com a utilização correta dos registradores

Constantes ou operandos imediatos

- Muitas vezes, valores são passados na instrução como **constantes**, e não como endereços de memória;
  - Nesses casos, é utilizado o modo de **endereçamento imediato**.
- Usando instruções comuns, teríamos de ler uma constante da memória para utilizá-la:

```
1. lw $t0, EndConstante4($s1) # EndConstante4 seria o endereço
2.                            # de memória da constante 4
3. add $s3, $s3, $t0
```

- Operandos como constantes ocorrem com bastante frequência
  - Oferecer instruções em que o operando seja uma constante:
    - Evita o uso de uma instrução *load (*uma leitura em memória desnecessária)
    - Torna as operações mais rápidas para serem executadas.

- **Princípio de projeto 3:**
  - Agilize os casos mais comuns

Constantes ou operandos imediatos

- A instrução imediata no caso de uma soma é o `add` imediato, ou `addi`:
  - `addi $s3, $s3, 4 # $s3 = $s3 + 4`
- Não existe um `subi`, pois pode ser executado com a constante negada.
  - `addi $s3, $s3, -4 # $s3 = $s3 - 4`

Assembly do MIPS

Assembly do MIPS				
Categoria	**Instrução**	**Exemplo**	**Significado**	**Comentários**
Aritmética	add	`add $s1, $s2, $S3`	$s1 = $s2 + $s3	3 operandos; Dados nos registradores.
	subtract	`sub $s1, $s2,$s3`	$s1 = $s2 - $s3	3 operandos; Dados nos registradores.
	Add immediate	`addi $s1, $s2, 4`	$s1 = $s2 - 4	3 operandos; Constante na instrução.
Tranf. de dados	load word	`lw $s1, 100($s2)`	$s1 = Memória[$s2 + 100]	Dados da memória para o registrador.
	store word	`sw $s1, 100($s2)`	Memória[$s2 + 100] = $s1	Dados do registrador para a memória.

REPRESENTANDO INSTRUÇÕES NO COMPUTADOR

Representando instruções no computador

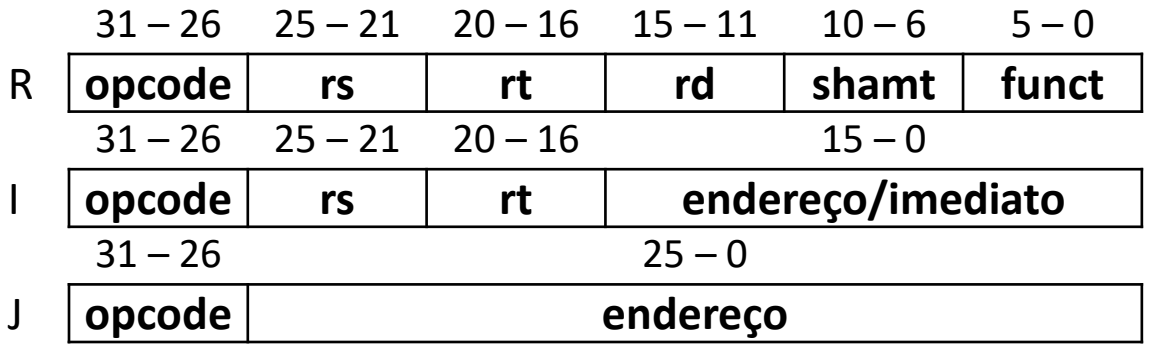
- A linguagem assembly utiliza mnemônicos (`lw`, `sw`, `add`, `sub`, etc);
  - Porém, instruções são representadas e executadas através de um conjunto de bits.
- Os registradores são parte de quase todas as instruções,
  - Convenção para mapear os nomes dos registradores em números binários:
    - `$s0` a `$s7` são mapeados nos registradores de 16 a 23;
    - `$t0` a `$t7` são mapeados nos registradores de 8 a 15;

Representando instruções no computador

Nome	Número do registrador	Uso	Valor preservado
$zero	0	Constante 0	n.a.
$at	1	Reservado para o montador	n.a.
$v0 - $v1	2-3	Valores retornados	não
$a0 - $a3	4-7	Argumentos	sim
$t0 - $t7	8-15	Temporários	não
$s0 - $s7	16-23	Valores salvos	sim
$t8 - $t9	24-25	Temporários	não
$k0 - $k1	26-27	Temporários SO	sim
$gp	28	Ponteiro global	sim
$sp	29	Ponteiro para pilha	sim
$fp	30	Ponteiro para quadro	sim
$ra	31	Endereço de retorno	sim

Representando instruções no computador

FORMATOS DE INSTRUÇÃO BÁSICOS (32 bits)



Representando instruções no computador

- O formato **R** (registrador):
  - **opcode** e **funct** combinados:
    - Dizem ao computador MIPS qual instrução dever ser executada;
  - **rs**: endereço do registrador que é o primeiro operando de origem
  - **rt**: endereço do outro operando de origem
  - **rd**: endereço do registrador que receberá o resultado
  - **shamt**: utilizado apenas em instruções de deslocamento (`ssl, srl`)

```
add $t0, $s1, $s2
```

31 – 26	*25 – 21*	*20 – 16*	*15 – 11*	*10 – 6*	*5 – 0*
opcode	**rs**	**rt**	**rd**	**shamt**	**funct**
0	17	18	8	0	32

Obs.: Valores em decimal.

Representando instruções no computador

- Algumas instruções precisam de **campos maiores** do que os presentes no formato R.
  - **Problemas de endereçamento**
  - Ex.: A instrução lw precisa especificar dois registradores e uma constante
    - Se o endereço tivesse apenas 5 bits do formato anterior, a constante estaria limitada a 32 (2^5).
  - **Conflito:** Instruções com o mesmo tamanho **Vs.** Formato de instrução único.
    - O **compromisso** escolhido pelos MIPS:
      - Todas as instruções com o mesmo tamanho:
      - Com diferentes formatos para os campos de diferentes tipos de instruções
  - Dessa forma temos:
    - O **formato R** (registrador)
    - O **formato I** (imediato)
      - Utilizado pelas instruções imediatas e de transferência de dados.

Representando instruções no computador

- O formato **I** (imediato):
  - **imediato (offset): 16 bits**
    - `lw` e `sw` podem acessar uma região de - 2^{15} até 2^{15} – 1 bytes (- 32.768 até 32.767 bytes) partir do endereço do registrador base (ou +/- 8.192 palavras).
    - A soma imediata (`addi`) é limitada a constantes não maiores do que 2^{15} (em magnitude).
  - **rt:**
    - Registrador de destino para `lw` e registrador de origem para `sw`
    - Registrador de destino para instruções imediatas (como `addi`)
  - **rs:**
    - Registrador de base para `lw` e `sw`
    - Registrador de origem para instruções imediatas (como `addi`)

31 – 26	*25 – 21*	*20 – 16*	*15 – 0*
opcode	**rs**	**rt**	**imediato/offset**
35	19	8	32

```
lw $t0, 32($s3)
```

Obs.: Valores em decimal.

Representando instruções no computador

- `lw $t0, 32($s3)`
  - 19 (`$s3`) é colocado no campo **rs**
  - 8 (`$t0`) é colocado no campo **rt**
  - 32 é colocado no campo de endereço:
  - O formato mudou: o campo **rt** especifica o registrador de destino, que recebe o resultado do `lw`.

31 – 26	*25 – 21*	*20 – 16*	*15 – 0*
opcode	**rs**	**rt**	**imediato/offset**
35	19	8	32

Exemplo: Assembly do MIPS para linguagem de máquina

- Considere que `$s1` armazena a base do *array* `A` e `$s2` corresponde a `h`:

```
1. A[300] = h + A[300];
```

- É compilado para:

```
1. lw $t0, 1200($s1) # reg. $t0 recebe A[300]
2. add $t0, $s2, $t0 # reg. $t0 recebe h+A[300]
3. sw $t0, 1200($s1) # armazena h+A[300] na mem.
```

	opcode	rs	rt	rd/ endereço	shamt/ endereço	funct/ endereço
`lw $t0, 1200($s1)`	35	17	8	1200		
`add $t0, $s2, $t0`	0	18	8	8	0	32
`sw $t0, 1200($s1)`	43	17	8	1200		

Obs.: Valores em decimal.

Exemplo: Assembly do MIPS para linguagem de máquina

- A instrução `lw` é representada por 35 no **opcode**.
- O registrador base 9 (`$t1`) é especificado no segundo campo (**rs**).
- O registrador de destino 8 (`$t0`) é especificado no terceiro campo (**rt**).
- O *offset* para selecionar `A[300]` (1200 = 300 × 4) aparece no campo final.
- A instrução `add` é especificada com **opcode** 0 e **funct** 32.
- A instrução `sw` é identificada com 43 no **opcode**.
- IMPORTANTE: Os valores estão representados em **decimal**, mas na vida real estão em **binário**.

	opcode	rs	rt	rd/ endereço	shamt/ endereço	funct/ endereço
`lw $t0, 1200($s1)`	35	17	8	1200		
`add $t0, $s2, $t0`	0	18	8	8	0	32
`sw $t0, 1200($s1)`	43	17	8	1200		

Obs.: Valores em decimal.

Exemplo: Assembly do MIPS para linguagem de máquina

** Valores em decimal*

	opcode	rs	rt	rd/ endereço	shamt/ endereço	funct/ endereço
`lw $t0, 1200($t1)`	35	9	8	1200		
`add $t0, $s2, $t0`	0	18	8	8	0	32
`sw $t0, 1200($t1)`	43	9	8	1200		

** Valores em binário*

	opcode	rs	rt	rd/ endereço	shamt/ endereço	funct/ endereço
`lw $t0, 1200($t1)`	100011	01001	01000	0000 0100 1011 0000		
`add $t0, $s2, $t0`	000000	10010	01000	01000	00000	100000
`sw $t0, 1200($t1)`	101011	01001	01000	0000 0100 1011 0000		

OPERAÇÕES LÓGICAS

Operações lógicas

- O projeto dos primeiros computadores se concentrava em palavras completas:
  - Entretanto, é útil atuar sobre campos de bits de uma palavra.
  - Operações lógicas são uteis para **empacotar** e **desempacotar** grupos de bits em palavras.

Operações lógicas	Operadores C	Instruções MIPS
Shift à esquerda	<<	sll
Shift à direita	>>	srl
AND bit a bit	&	and, andi
OR bit a bit	|	or, ori
NOT bit a bit	~	nor

Operações lógicas

- Operações de deslocamento (*shifts*)
  - Movem todos os bits de uma palavra para esquerda ou para direita;
    - Preenche com zero os bits que ficaram vazios.
  - O deslocamento lógico à **esquerda** de *i* bits equivale a multiplicar por 2^i.
  - O deslocamento lógico à **direita** de *i* bits equivale a dividir por 2^i.

Operações lógicas

- *Shift left logical* (deslocamento lógico à esquerda):
  - `sll $t2, $s0, 4  # reg $t2 = reg $s0 << 4 bits`
  - O campo **shamt** (*shift amount*) da instrução MIPS:
    - Usado nas instruções de deslocamento.
  - `sll` é codificada com zeros nos campos **op** e **funct;**
    - **rt** contém `$t2`, **rs** contém `$s0` e **shamt** contém 4.
    - **rd** não é utilizado (0).

31 – 26	*25 – 21*	*20 – 16*	*15 – 11*	*10 – 6*	*5 – 0*
opcode	**rs**	**rt**	**rd**	**shamt**	**funct**
0	16	10	0	4	0

Operações lógicas

- *Shift right logical* (deslocamento lógico à direta):
  - `srl $t2, $s0, 4  # reg $t2 = reg $s0 >> 4 bits`
  - Semelhante ao `sll`, porém desloca os bits à direita.

31 – 26	*25 – 21*	*20 – 16*	*15 – 11*	*10 – 6*	*5 – 0*
opcode	**rs**	**rt**	**rd**	**shamt**	**funct**
0	16	10	0	4	2

Operações lógicas

- A operação AND
  - Útil para isolar bits de uma palavra (operações com máscara)
    - `and $t0, $t1, $t2  # $t0 = $t1 & $t2`
- Operação OR:
  - Pode ser utilizado para colocar um bit em 1 em um grupo de bits
    - `or $t0, $t1, $t2  # $t0 = $t1 | $t2`

- `$t1          = 0000 0101 1100 1011 1110 0010 1111 0001`
- `$t2          = 1100 1011 1110 0010 1111 0001 0000 0101`
- `$t1 and $t2 = 0000 0001 1100 0010 1110 0000 0000 0001`
- `$t1 or $t2  = 1100 1111 1110 1011 1111 0011 1111 0101`

Operações lógicas

- Os projetistas do MIPS decidiram incluir a instrução NOR no lugar de NOT:
  - Se um operando for zero, a instrução é equivalente a NOT:
    - A NOR 0 = NOT (A OR 0) = NOT (A)
    - `nor $t0, $t1, $t3      # $t0 = ~($t1 | $t3)`
    - `nor $t0, $t1, $zero    # $t0 = ~$t1`

- O MIPS oferece instruções lógicas para trabalhar com constantes (modo de endereçamento imediato):
  - AND imediato (`andi`);
  - OR imediato (`ori`).

INSTRUÇÕES PARA TOMADA DE DECISÃO

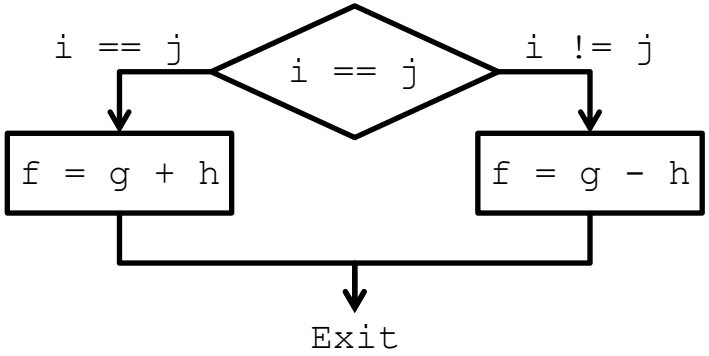
Instruções para tomadas de decisão

- Capacidade de tomar decisões:
  - Diferencia um computador de uma simples calculadora
  - Em linguagens de alto nível:
    - `if`, `else`, `goto`, ...
- Instruções de desvio condicional no MIPS
  - *Branch if equal* (desvie se igual)
    - `beq registrador1, registrador2, L1`
    - Se o valor no `registrador1` for igual ao valor no `registrador2`:
      - Desvie para a instrução rotulada por `L1`;
  - *Branch if not equal* (desvie se não for igual)
    - `bne registrador1, registrador2, L1`
    - Se o valor no `registrador1` não for igual ao valor no `registrador2`:
      - Desvie para a instrução rotulada por `L1`;
  - O rótulo `L1` é alterado pelo compilador por um endereços de memória.

Instruções para tomadas de decisão

- Considere a sequencia de código em linguagem C:

```
1. if (i == j)
2.     f = g + h;
3. else
4.     f = g - h;
```



- Em assembly do MIPS:

```
1.      bne $s3, $s4, Else # Se $s3 != $s4, desvie para Else
2.      add $s0, $s1, $s2
3.      j Exit      # Desvie para Exit, incondicionalmente
4.Else: sub $s0, $s1, $s2
5.Exit: # o código continua...
```

- A instrução `j` implementa o desvio incondicional (*jump*).

a02.ex03.asm

Instruções para tomadas de decisão

- Laços de repetição são essenciais para construção de programas:
  - `while`, `do until`, etc.
- Compilando um loop while em C:

```
1.while (save[i] == k)
2.   i += 1;
```

- `i` e `k` correspondam aos registradores `$s3` e `$s5` e a base do vetor `save` está em `$s6`.

```
1.Loop: sll  $t1, $s3, 2     # $t1 = 4 * i
2.      add  $t1, $t1, $s6   # $t1 = endereço de save[i]
3.      lw   $t0, 0($t1)     # $t0 = save[i]
4.      bne  $t0, $s5, Exit  # Vá para Exit se save[i] <> k
5.      addi $s3, $s3, 1     # i += 1
6.      j Loop
7.Exit: # o código continua...
```

a02.ex05.asm

Instruções para tomadas de decisão

- `beq` e `bne` permitem apenas comparar se dois valores são iguais ou não.
  - Entretanto, é necessário comparar se um valor é menor ou maior do o outro

- *Set on less than* (atribuir se menor que):
  - `slt $t0, $s3, $s4`
    - `$t0 recebe` 1 se o valor em `$s3` for menor do que o em `$s4`.

Instruções para tomadas de decisão

- Comparando com constantes
  - Operadores constantes são populares nas comparações;
  - Para comparar com zero, sempre usamos o registrador `$zero` (que sempre contém 0);
  - Para comparar com outros valores, usamos `slt` com endereçamento imediato (`slti`):
    - `slti $t0, $s2, 10 # $t0 = 1 se $s2 < 10`

- Interface hardware/software
  - Os compiladores MIPS utilizam as instruções `slt`, `slti`, `beq`, `bne` e o valor fixo 0 (`$zero`) para criar todas as condições relativas:
    - `==; !=; <; >; <=; >=.`

Instruções para tomadas de decisão

C	Assembly

`if (a<b) {`
`  < instruções T >`
`} else {`
`  < instruções F >`
`}`
`// a em $s0 e b em $s1.`
	`  slt $t0, $s0, $s1  # $t0=1 se $s0<$s1 (a<b)`
	`  beq $t0, $zero, ELSE  # Desvia se $t0=0`
	`  < bloco de instruções T >`
	`  j END  # Evita o bloco F.`
	`ELSE: < bloco de instruções F >`
	`END:`

`if (a>b) { // if (b<a)`
`  < instruções T >`
`} else {`
`  < instruções F >`
`}`
`// T: Verdadeiro`
`// F: Falso`
	`  slt $t0, $s1, $s0  # $t0=1 se $s1<$s0 ((b<a) == (a>b))`
	`  beq $t0, $zero, ELSE # Desvia se $t0==0`
	`  < bloco de instruções T >`
	`  j END  # Evita o bloco F.`
	`ELSE: < bloco de instruções F >`
	`END:`

`if (a>=b) {`
`  < instruções T >`
`} else {`
`  < instruções F >`
`}`
	`  slt $t0, $s0, $s1  # $t0=1 se $s0<$s1 ((a>=b) == !(a<b))`
	`  **bne** $t0, $zero, ELSE  # Desvia se $t0!=0 ($t0==1)`
	`  < bloco de instruções T >`
	`  j END  # Evita o bloco F.`
	`ELSE: < bloco de instruções F >`
	`END:`

`if (a<=b) {`
`  < instruções T >`
`} else {`
`  < instruções F >`
`}`
	`  slt $t0, $s1, $s0  # $t0=1 se $s1<$s0 ((a<=b) == !(a>b))`
	`  **bne** $t0, $zero, ELSE # Desvia se $t0!=0 ($t0==1)`
	`  < bloco de instruções T >`
	`  j END  # Evita o bloco F.`
	`ELSE: < bloco de instruções F >`
	`END:`

a02.ex04.asm

Instruções para tomadas de decisão

- Bloco básico:
  - Uma sequencia de instruções sem desvios (exceto, possivelmente no final) e sem destinos de desvio ou rótulos de desvio (exceto, possivelmente, no início)
  - *Uma das primeiras fases da compilação é desmembrar o programa em blocos básicos*.

Instruções para tomadas de decisão

```
1. int f(int a, int b) {
2.     int c, i, j;
3.     c = 5;
4.     for(i=0; i<a; i++){
5.         for(j=0; j<b; j++){
6.             if(i < c)
7.                 c = j;
8.             printf("%d, %d, %d", c, i, j);
9.         }
10.        printf("%d", i);
11.    }
12.    return c;
13.}
```

3

2

1

0

Assembly do MIPS

Categoria	Instrução	Exemplo	Significado
Aritmética	Add	`add $s1, $s2, $s3`	$s1 = $s2 + $s3
	Subtract	`sub $s1, $s2, $s3`	$s1 = $s2 - $s3
	Add immediate	`addi $s1, $s2, 4`	$s1 = $s2 - 4
Transf. de dados	Load word	`lw $s1, 100($s2)`	$s1 = Memória[$s2 + 100]
	Store word	`sw $s1, 100($s2)`	Memória[$s2 + 100] = $s1
Lógica	And	`and $s1, $s2, $s3`	$s1 = $s2 & $s3
	Or	`or $s1, $s2, $s3`	$s1 = $s2 | $s3
	Nor	`nor $s1, $s2, $s3`	$s1 = ~($s2 | $s3)
	And immediate	`andi $s1, $s2, 100`	$s1 = $s2 & 100
	Or immediate	`ori $s1, $s2, 100`	$s1 = $s2 | 100
	Shift left logical	`sll $s1, $s2, 10`	$s1 = $s2 << 10
	Shift righ logical	`srl $s1, $s2, 10`	$s1 = $s2 >> 10
Desvio condicional	Branch on equal	`beq $s1, $s2, L`	If ($s1 == $s2) go to L
	Branch on not equal	`bnq $s1, $s2, L`	If ($s1 != $s2) go to L
	Set on less than	`slt $s1, $s2, $s3`	If ($s2 < $s3) $s1 = 1 else $s1 = 0
	Set on less than immediate	`slti $s1, $s2, 100`	If ($s2 < 100) $s1 = 1 else $s1 = 0
Desvio incondicional	Jump	`j L`	Go to L

SUPORTE PARA PROCEDIMENTOS NO HARDWARE DO COMPUTADOR

Suporte para procedimentos

- Procedimentos ou funções (sete etapas):
  1. Colocar parâmetros em um lugar onde o procedimento possa acessá-lo;
  2. Transferir o controle para o procedimento;
  3. Adquirir os recursos de armazenamento necessários para o procedimento;
  4. Realizar a tarefa desejada;
  5. Colocar o valor de retorno em um local onde o programa que o chamou possa acessá-lo;
  6. Restaurar o contexto de execução do programa;
  7. Retornar o controle para o ponto de origem, pois um procedimento pode ser chamado de vários pontos de um programa.

Suporte para procedimentos

- Registradores do MPIS utilizados para chamada de procedimentos:
  - `$a0` - `$a3`: Quatro registradores de argumento
    - Passagem de parâmetros
  - `$v0` - `$v1`: Dois registradores de valor
    - Valores de retorno
  - `$ra`: Registrador de endereço de retorno
    - Para retornar ao ponto de origem do programa que efetuou a chamada;

- O *assembly* do MIPS inclui uma instrução apenas para chamada dos procedimentos:
  - `jal` (jump-and-link): Desvia para um endereço e simultaneamente salva o endereço da instrução seguinte no registrador `$ra`.

Suporte para procedimentos

- Instrução *jump-and-link* (`jal`)
  - Instrução do MIPS inclui usada apenas para chamada dos procedimentos
  - `jal LabelProc`
  - Desvia para um endereço e simultaneamente salva o endereço da instrução seguinte no registrador `$ra` (endereço de retorno);
    - Salva PC + 4 no registrador `$ra` (*link*)
    - Desvia (*jump*) para o procedimento no endereço indicado pelo rótulo `LabelProc`.
  - Contador de programa ou PC (*program counter*)
    - Registrador que mantem o endereço da próxima instrução a ser executada

Suporte para procedimentos

- instrução *jump register* (`jr`):
  - Executa um desvio incondicional para o endereço especificado no registrador
  - Pensado para suporta à procedimentos no MIPS
  - `jr $ra`
    - O programa que chama o procedimento, coloca os valores de parâmetro em `$a0-$a3` e utiliza `jal X` para desviar para o procedimento X (*callee*).
    - O procedimento `X` realiza as suas operações, coloca os resultados em `$v0-$v1` e retorna o controle para o *caller* usando `jr $ra`.

Suporte para procedimentos

- Quando um compilador precisa de mais registradores para um procedimento do que os disponíveis:
  - Utiliza-se a pilha (*stack*)
- Pilha:
  - Uma LISTA em que o último que entra é o primeiro que sai:
    - FILO (First In → Last Out)
  - O ponteiro de pilha (*stack pointer* – `$sp`) armazena o endereço do topo da pilha.
    - É ajustado em uma palavra para cada registrador salvo ou restaurado
      - Push → insere itens
      - Pop → remove itens
  - As pilhas crescem de endereços maiores para menores.

Suporte para procedimentos

- Compilando um procedimento em C:

```
1.  int exemplo_folha (int g, int h, int i, int j){
2.     int f;
3.     f = ( g + h ) - ( i + j );
4.     return f;
5.  }
```

- As variáveis de parâmetro `g`, `h`, `i` e `j` correspondem ao registradores de argumento `$a0`–`$a3` e `f` corresponde a `$s0`

- Procedimento folha não possuí chamadas à funções.
  - Não é necessário em pilhar `$ra`

Suporte para procedimentos

```
1.  int exemplo_folha (int g, int h, int i, int j){ // $a0 - $a3
2.    int f; // $s0
3.    f = ( g + h ) - ( i + j );
4.    return f;
5.  }
```

```
1.  exemplo_folha: addi $sp, $sp, -4    # ajusta a pilha (1 itens)
2.                 sw $s0, 0($sp)       # empilha registrador $s0
3.                 add $t0, $a0, $a1    # $t0 contém g+h
4.                 add $t1, $a2, $a3    # $t1 contém i+j
5.                 sub $s0, $t0, $t1    # f = (g+h) - (i+j)
6.                 add $v0, $s0, $zero  # copia f para reg. de retorno
7.                 lw $s0, 0($sp)       # desempilha registrador $s0
8.                 addi $sp, $sp, 4     # exclui 1 itens da pilha
9.                 jr $ra               # Retorna para o endereço em $ra
```

a02.ex06.asm

Suporte para procedimentos

```
1.  int exemplo_folha (int g, int h, int i, int j){ // $a0 - $a3
2.    int f; // $s0
3.    f = ( g + h ) - ( i + j );
4.    return f;
5.  }
```

```
1.  exemplo_folha: addi $sp, $sp, -4    # ajusta a pilha (1 itens)
2.                 sw $s0, 0($sp)       # empilha registrador $s0
3.                 add $t0, $a0, $a1    # $t0 contém g+h
4.                 add $t1, $a2, $a3    # $t1 contém i+j
5.                 sub $s0, $t0, $t1    # f = (g+h) - (i+j)
6.                 add $v0, $s0, $zero  # copia f para reg. de retorno
7.                 lw $s0, 0($sp)       # desempilha registrador $s0
8.                 addi $sp, $sp, 4     # exclui 1 itens da pilha
9.                 jr $ra               # Retorna para o endereço em $ra
```

Suporte para procedimentos

- O software do MIPS separa 18 dos registradores em dois grupos (por convenção):
  - `$t0-$t9`: 10 registradores temporários
    - Não são preservados pelo procedimento *chamado*
  - `$s0-$s7`: 8 registradores que precisam ser preservados em uma chamada
    - Se forem usados, o procedimento *chamado* os salva e restaura

- Procedimentos aninhados:
  - A solução é empilhar os valores dos registradores que vão ser utilizados
    - Procedimento *chamador* (*caller*): empilha (`$a0-$a3`) e (`$t0-$t7`);
    - Procedimento *chamado* (*callee*): empilha `$ra` e registradores salvos usados por ele (`$s0-$s7`).

Suporte para procedimentos

```
1.    int func2(int m) { // $a0
2.       int a; // $s0
3.       a = m - 1;
4.       return a;
5.    }
6.    int func1(int m, int n){ // $a0 e $a1
7.        int c, d; // $s0 e  $s1
8.        c = m + n;
9.        d = func2(c);
10.       return d;
11.   }
12.   int main() {
13.      int a, b; // $s0 e $s1
14.      a = func1(2, 3);
15.      printf("%d", a);
16.      return 0;
17.   }
```

```
1.    main:   # Alimenta reg. de argumentos (func1)
2.            addi $a0, $zero, 2
3.            addi $a1, $zero, 3
4.            jal  func1
5.            add  $s0, $zero, $v0
6.            # Print
7.            li $v0, 1
8.            move $a0, $s0 # pseudo-instrução
9.            syscall
10.           # Encerra o programa
11.           li $v0, 10 # pseudo-instrução
12.           syscall
```

```
1.    func1:  # Função chamada empilha $ra (somente se não-folha) e $sX.
2.            addi $sp, $sp, -12      # ajusta a pilha (3 itens)
3.            sw   $ra, 8($sp)        # $ra - funçao chamada empilha
4.            sw   $s0, 4($sp)        # c - função chamada empilha
5.            sw   $s1, 0($sp)        # d - função chamada empilha
6.            add  $s0, $a0, $a1      # Código da função
7.            # Função chamadora (não-folha) empilha $aX e $tX (Somente se necessário).
8.            # Não é necessário, não vamos utilizar $a0 e $a1 após a chamada de func2
9.            addi $sp, $sp, -8
10.           sw   $a0, 4($sp)        # Fizemos apenas para demonstração.
11.           sw   $a1, 0($sp)
12.           add  $a0, $zero, $s0    # Alimenta registradores e argumentos
13.           jal  func2
14.           add  $s1, $zero, $v0
15.           lw  $a1, 0($sp)         # Desempilha $sX e $tX, caso tenha empilhado
16.           lw  $a0, 4($sp)
17.           addi $sp, $sp, 8        # Libera espaço na pilha
18.           add $v0, $zero, $s1
19.           lw   $s1, 0($sp)        # Desempilha
20.           lw   $s0, 4($sp)
21.           lw   $ra, 8($sp)
22.           addi $sp, $sp, 12       # Libera espaço na pilha
23.           jr  $ra                 # Retorna o controle para a função chamadora
24.   func2: addi $sp, $sp, -4        # ajusta a pilha (3 itens)
25.           sw   $s0, 0($sp)
26.           addi  $s0, $a0, -1
27.           add $v0, $zero, $s0
28.           lw  $s0, 0($sp)
29.           add $sp, $sp, 4
30.           jr $ra
```

a02.ex07.asm

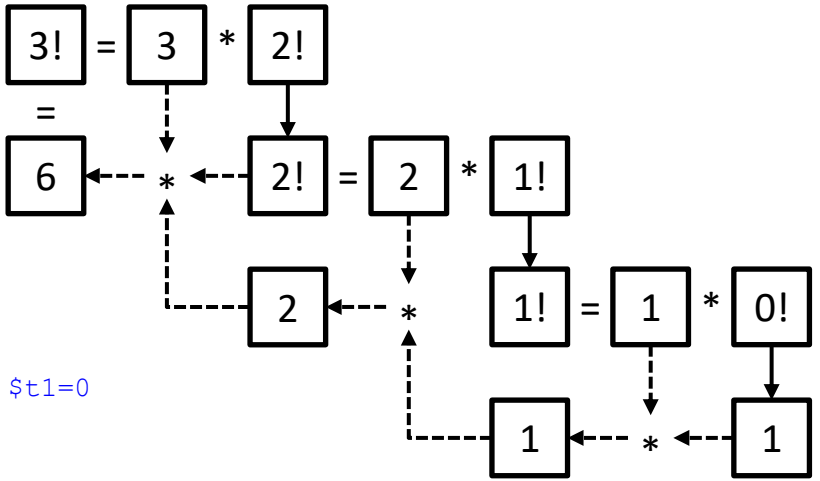
a02.ex07a.asm

Versão mais complexa!

Suporte para procedimentos

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

```
1.      fact: addi $sp, $sp, -8      # ajusta pilha para 2 itens
2.            sw $ra, 4($sp)         # salva endereço de retorno
3.            sw $a0, 0($sp)         # salva o argumento n
4.            slti $t0, $a0, 1       # se (n<1) → $t0=1; se n>=1 → $t1=0
5.            beq $t0, $zero, ELSE   # se n >= 1, desvia para ELSE
6.            addi $v0, $zero, 1     # prepara o "retorna 1"
7.            addi $sp, $sp, 8       # retira dois itens da pilha
8.            jr $ra                 # retorna
9.      ELSE: addi $a0,$a0, -1       # argumento recebe n-1
10.           jal fact               # chama fact com n-1
11.           lw $a0, 0($sp)         # retorna de jal: restaura o arg. n
12.           lw $ra, 4($sp)         # restaura o endereço de retorno
13.           addi $sp, $sp, 8       # ajusta pilha para remover 2 itens
14.           mul $v0, $a0, $v0      # calcula n * fact(n-1)
15.           jr $ra                 # retorna para o procedimento que chamou
```



a02.ex08.asm

Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	60
$ra	?
$a0	3
$t0	X
$v0	X

3! = 3 * 2!

?: Endereço de retorno da função chamadora.

X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
	44	
	48	
	52	
	56	
$sp	60	X
	64	X
	68	X
	72	X
	76	X

Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	?
$a0	3
$t0	X
$v0	X

3! = 3 * 2!

X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
	44	
	48	
$sp	52	
↑	56	
$sp	60	X
	64	X
	68	X
	72	X
	76	X

Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	?
$a0	3
$t0	X
$v0	X

3! = 3 * 2!

X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
	44	
	48	
$sp	52	
	56	**?**
	60	X
	64	X
	68	X
	72	X
	76	X

Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	?
$a0	3
$t0	X
$v0	X

3! = 3 * 2!

X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
	44	
	48	
$sp	52	**3**
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	?
$a0	3
$t0	**0** # pois $a0 >= 1
$v0	X

3! = 3 * 2!

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
	44	
	48	
$sp	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	?
$a0	3
$t0	0
$v0	X

3! = 3 * 2!

X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
	44	
	48	
$sp	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	?
$a0	**2** # $a0 = 3 – 1
$t0	0
$v0	X

3! = 3 * 2!

X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
	44	
	48	
$sp	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

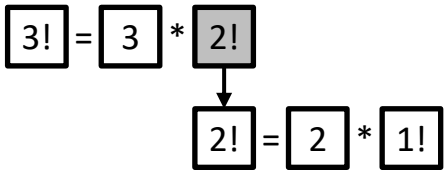
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	**110**
$a0	2
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
	44	
	48	
$sp	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

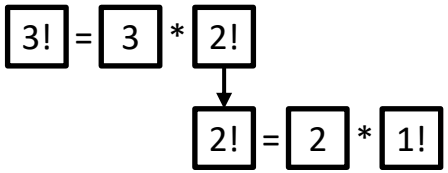
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	**44**
$ra	110
$a0	2
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
$sp	44	
	48	
$sp	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

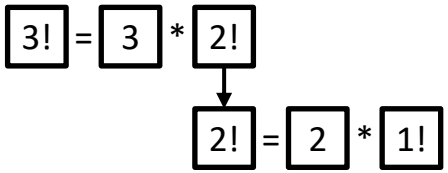
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	110
$a0	2
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
$sp	44	
	48	**110**
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

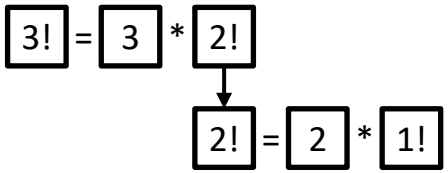
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	110
$a0	2
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
$sp	44	**2**
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

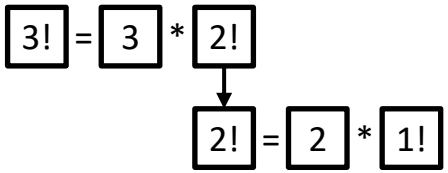
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	110
$a0	2
$t0	**0** # pois $a0 >= 1
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
$sp	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

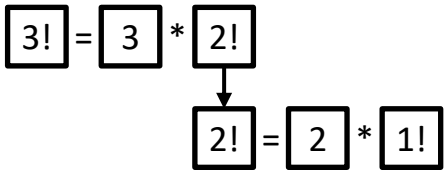
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	110
$a0	2
$t0	0
$v0	X



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
$sp	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

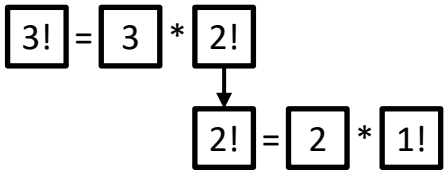
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	110
$a0	**1** # $a0 = 2 – 1
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
$sp	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

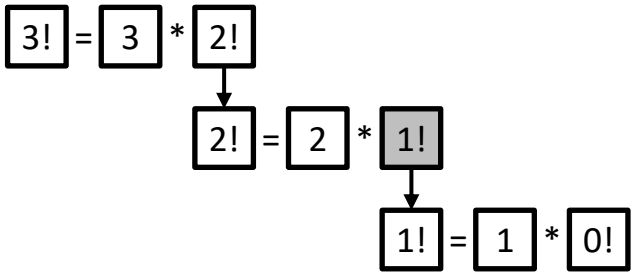
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	**110**
$a0	1
$t0	0
$v0	X



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
	36	
	40	
$sp	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

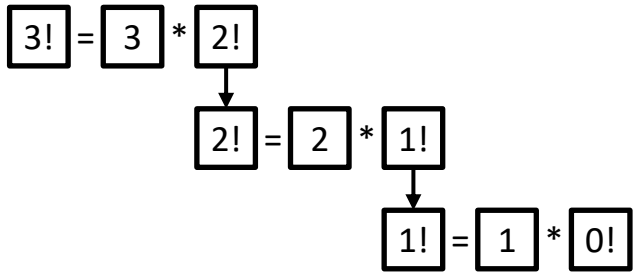
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	**36**
$ra	110
$a0	1
$t0	0
$v0	X



Memória Principal

	End.	Instruções e Dados
$sp	36	
	40	
$sp	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

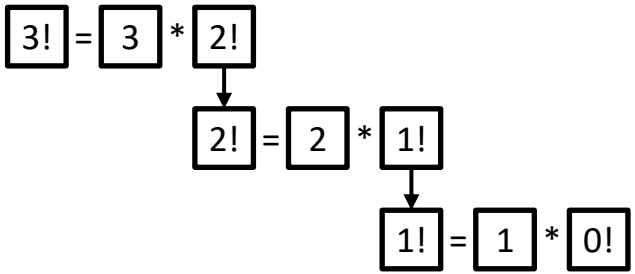
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	36
$ra	110
$a0	1
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
$sp	36	
	40	**110**
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

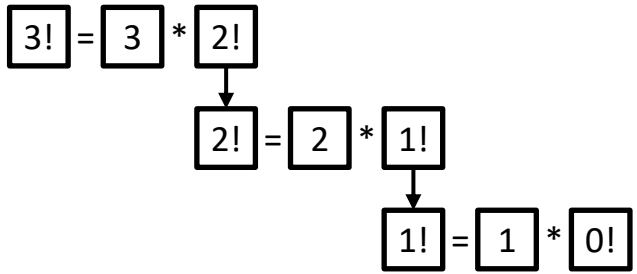
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	36
$ra	110
$a0	1
$t0	0
$v0	X



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
$sp	36	**1**
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

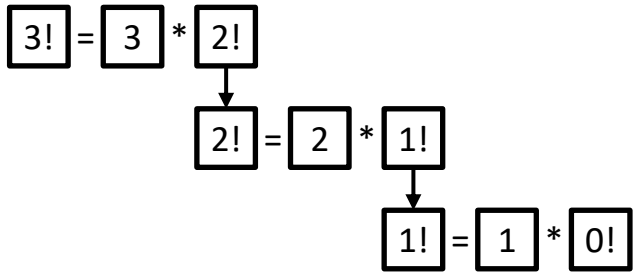
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	36
$ra	110
$a0	1
$t0	**0** # pois $a0 >= 1
$v0	X



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
$sp	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

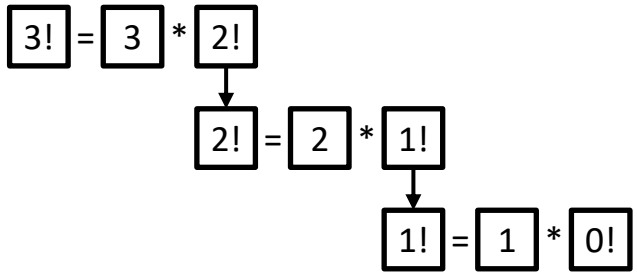
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	36
$ra	110
$a0	1
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
$sp	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

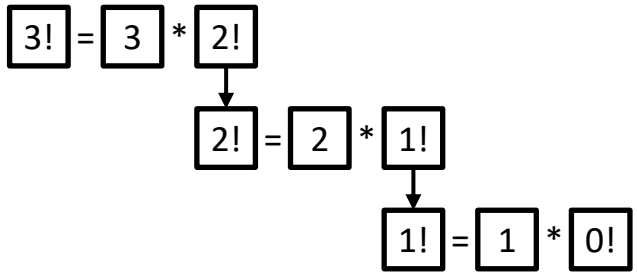
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	36
$ra	110
$a0	**0** # $a0 = 1 - 1
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
$sp	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

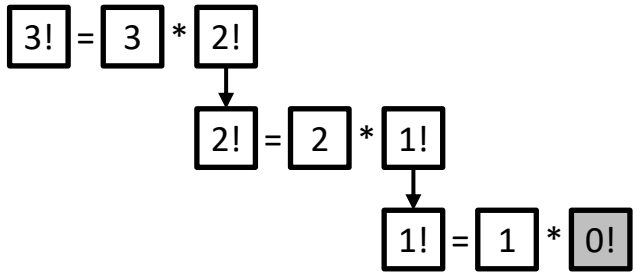
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	36
$ra	**110**
$a0	0
$t0	0
$v0	X



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	
	32	
$sp	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

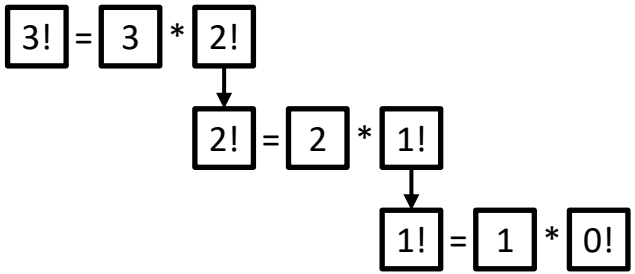
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	**28**
$ra	110
$a0	0
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
$sp	28	
↑	32	
$sp	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

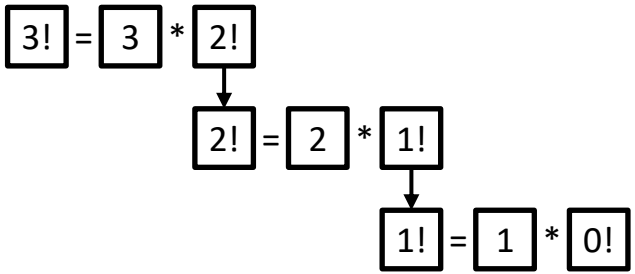
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	28
$ra	110
$a0	0
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
$sp	28	
	32	**110**
	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

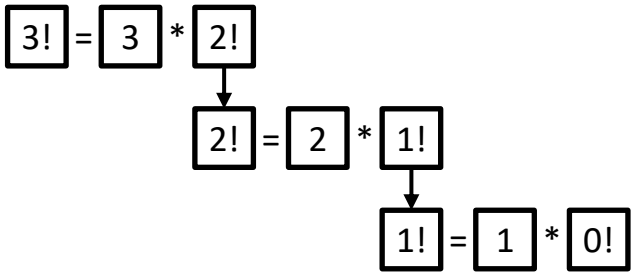
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	28
$ra	110
$a0	0
$t0	0
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
$sp	28	**0**
	32	110
	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

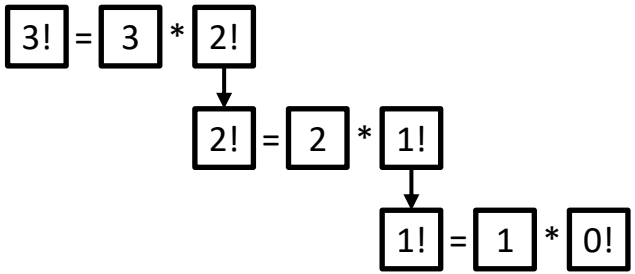
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	28
$ra	110
$a0	0
$t0	**1** # pois $a0 < 1
$v0	X



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
$sp	28	**0**
	32	110
	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

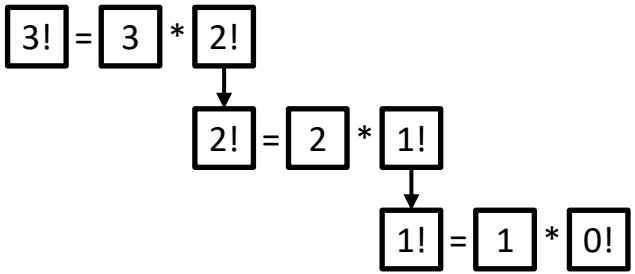
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	28
$ra	110
$a0	0
$t0	1
$v0	X



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
$sp	28	0
	32	110
	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

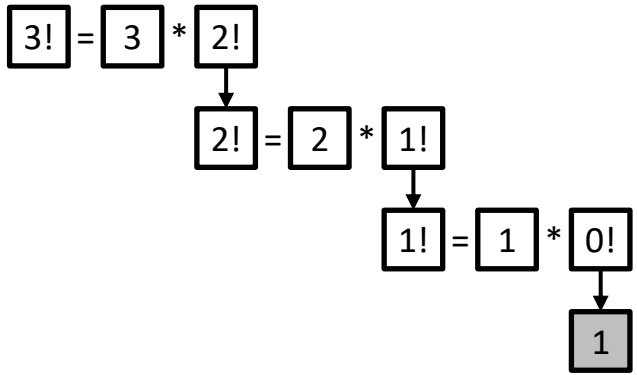
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	28
$ra	110
$a0	0
$t0	1
$v0	**1**



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
$sp	28	**0**
	32	110
	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

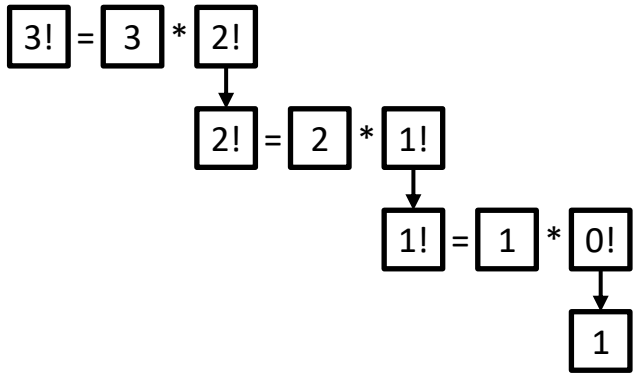
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	**36**
$ra	110
$a0	0
$t0	1
$v0	1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
$sp	28	~~0~~
↓	32	~~110~~
$sp	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

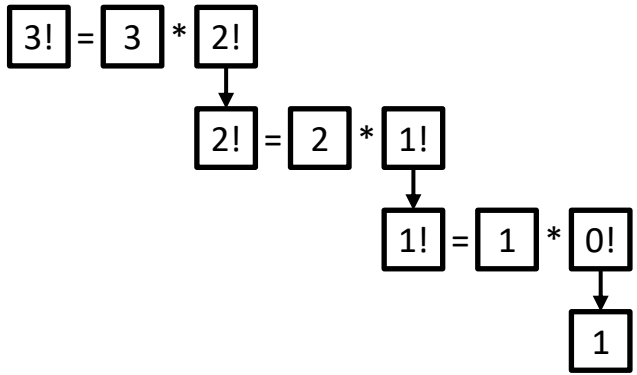
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	36
$ra	110
$a0	0
$t0	1
$v0	1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
$sp	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

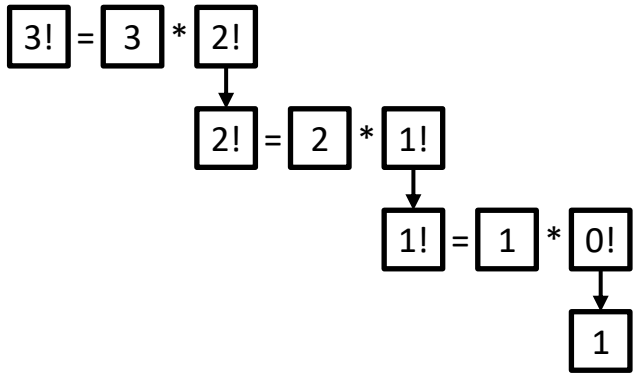
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	36
$ra	110
$a0	**1**
$t0	1
$v0	1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
$sp	**36**	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

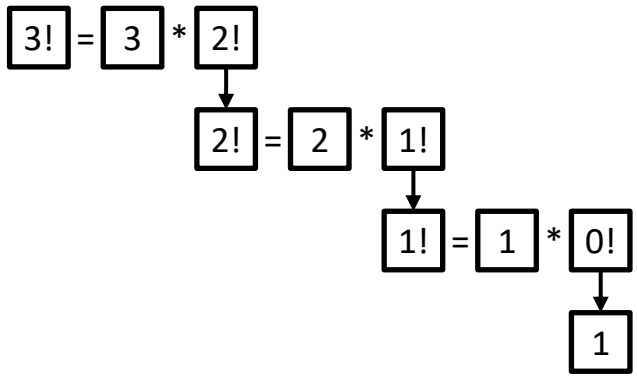
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	36
$ra	**110**
$a0	1
$t0	1
$v0	1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
$sp	36	1
	40	110
	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

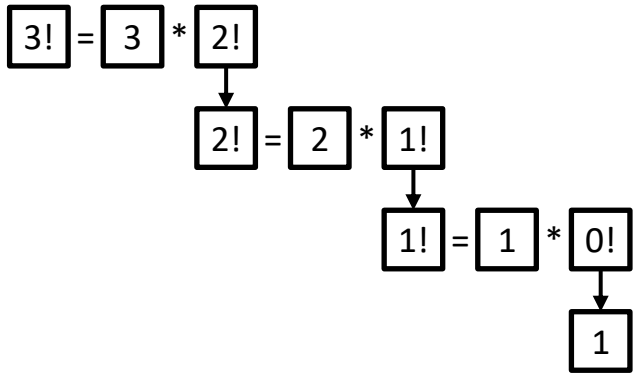
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	**44**
$ra	110
$a0	1
$t0	1
$v0	1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
$sp	36	~~1~~
↓	40	~~110~~
$sp	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

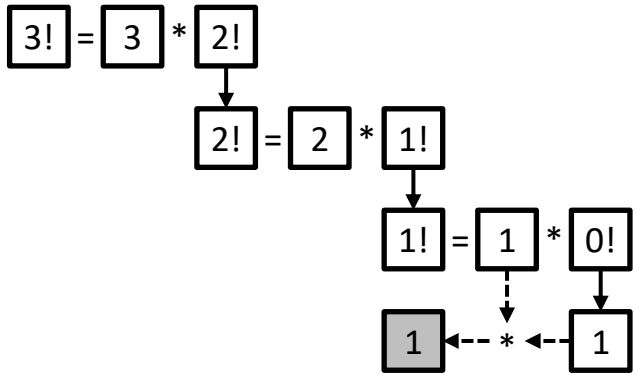
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	110
$a0	1
$t0	1
$v0	**1** # 1 * 1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
	36	~~1~~
	40	~~110~~
$sp	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

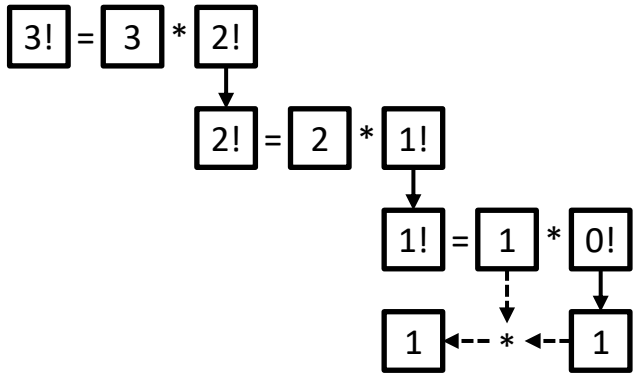
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	110
$a0	1
$t0	1
$v0	1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
	36	~~1~~
	40	~~110~~
$sp	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

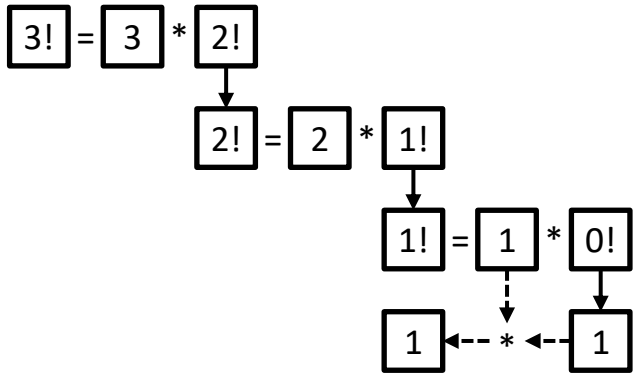
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	110
$a0	**2**
$t0	1
$v0	1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
	36	~~1~~
	40	~~110~~
$sp	**44**	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

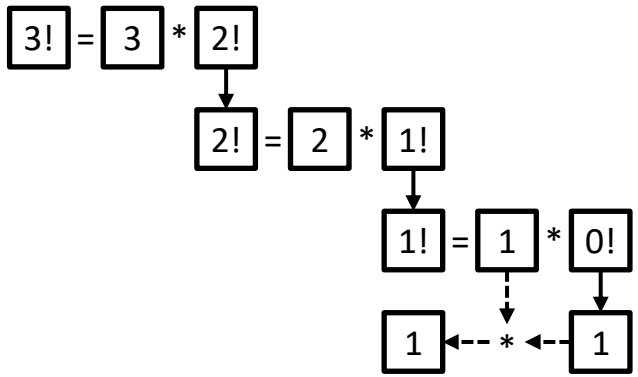
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	44
$ra	**110**
$a0	2
$t0	1
$v0	1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
	36	~~1~~
	40	~~110~~
$sp	44	2
	48	110
	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

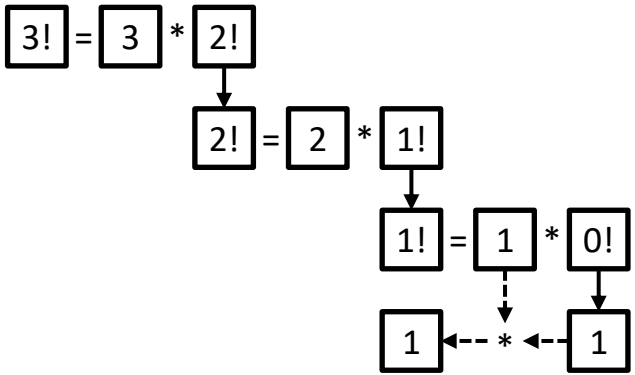
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	**52**
$ra	110
$a0	2
$t0	1
$v0	1



Memória Principal

End.	Instruções e Dados
0	
4	
8	
12	
16	
20	
24	
28	~~0~~
32	~~110~~
36	~~1~~
40	~~110~~
$sp → 44	~~2~~
48	~~110~~
$sp → 52	3
56	?
60	X
64	X
68	X
72	X
76	X

X: algum dado gravado.

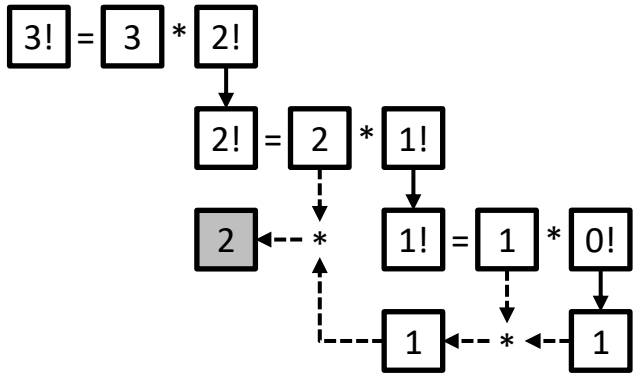
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	110
$a0	2
$t0	1
$v0	**2** # 2 * 1



Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
	36	~~1~~
	40	~~110~~
	44	~~2~~
	48	~~110~~
$sp	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

X: algum dado gravado.

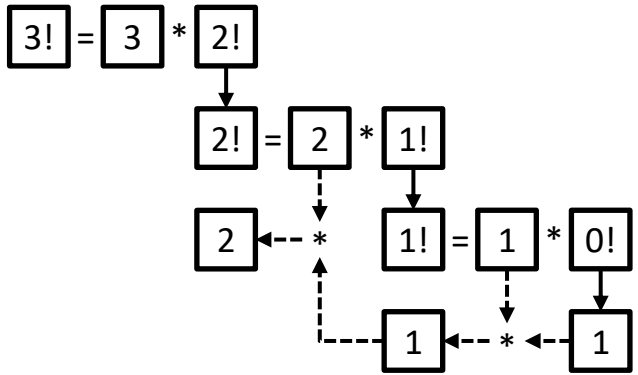
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	110
$a0	2
$t0	1
$v0	2



X: algum dado gravado.

Memória Principal

	End.	Instruções e Dados
	0	
	4	
	8	
	12	
	16	
	20	
	24	
	28	~~0~~
	32	~~110~~
	36	~~1~~
	40	~~110~~
	44	~~2~~
	48	~~110~~
$sp	52	3
	56	?
	60	X
	64	X
	68	X
	72	X
	76	X

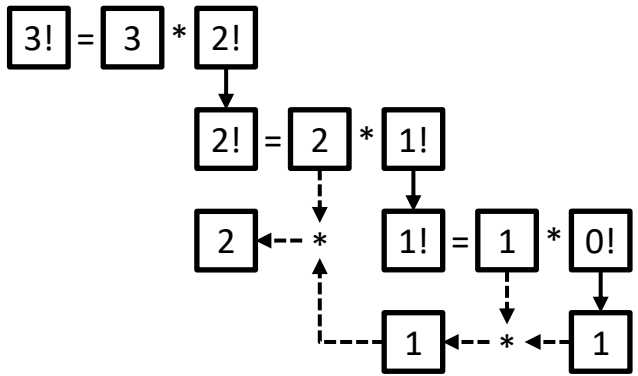
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	110
$a0	**3**
$t0	1
$v0	2



X: algum dado gravado.

Memória Principal

End.	Instruções e Dados
0	
4	
8	
12	
16	
20	
24	
28	~~0~~
32	~~110~~
36	~~1~~
40	~~110~~
44	~~2~~
48	~~110~~
$sp **52**	3
56	?
60	X
64	X
68	X
72	X
76	X

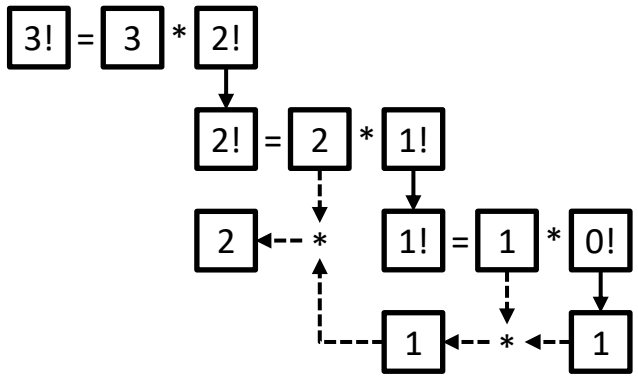
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	52
$ra	**?**
$a0	3
$t0	1
$v0	2



X: algum dado gravado.

Memória Principal

End.	Instruções e Dados
0	
4	
8	
12	
16	
20	
24	
28	~~0~~
32	~~110~~
36	~~1~~
40	~~110~~
44	~~2~~
48	~~110~~
$sp 52	3
56	?
60	X
64	X
68	X
72	X
76	X

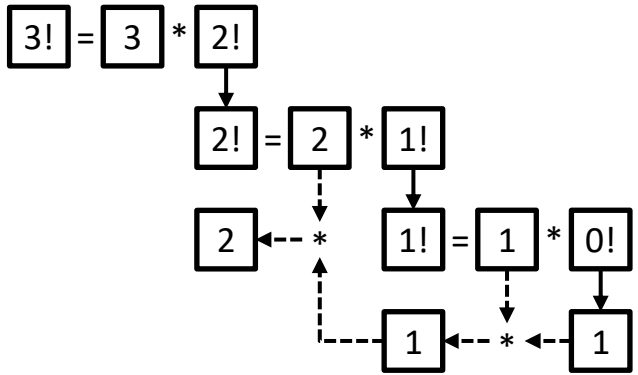
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	**60**
$ra	?
$a0	3
$t0	1
$v0	2



Memória Principal

End.	Instruções e Dados
0	
4	
8	
12	
16	
20	
24	
28	~~0~~
32	~~110~~
36	~~1~~
40	~~110~~
44	~~2~~
48	~~110~~
52	~~3~~
56	~~?~~
60	X
64	X
68	X
72	X
76	X

$sp → $sp

X: algum dado gravado.

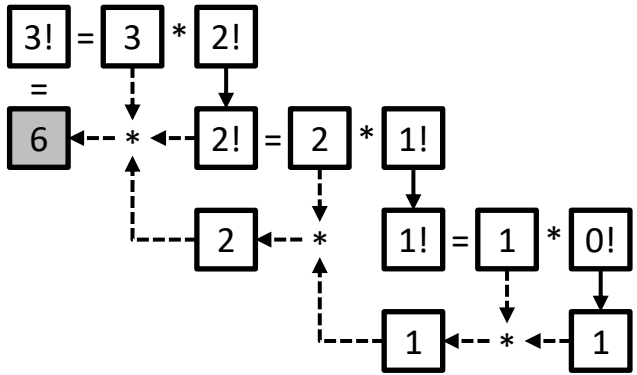
Suporte para procedimentos

```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	60
$ra	?
$a0	3
$t0	1
$v0	**6** # 3 * 2



Memória Principal

End.	Instruções e Dados
0	
4	
8	
12	
16	
20	
24	
28	~~0~~
32	~~110~~
36	~~1~~
40	~~110~~
44	~~2~~
48	~~110~~
52	~~3~~
56	~~?~~
60	X
64	X
68	X
72	X
76	X

X: algum dado gravado.

Suporte para procedimentos

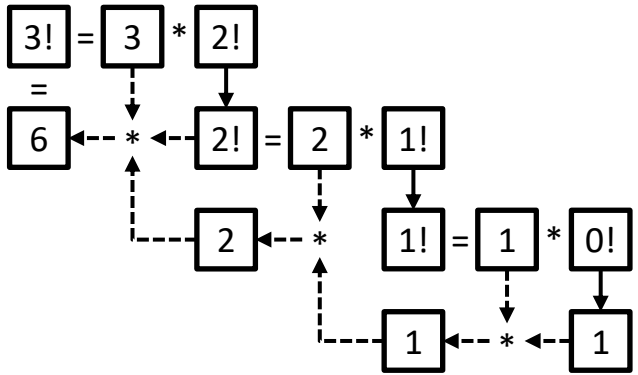
```
100.   fact: addi $sp, $sp, -8
101.         sw $ra, 4($sp)
102.         sw $a0, 0($sp)
103.         slti $t0, $a0, 1
104.         beq $t0, $zero, ELSE
105.         addi $v0, $zero, 1
106.         addi $sp, $sp, 8
107.         jr $ra
108.   ELSE: addi $a0,$a0, -1
109.         jal fact
110.         lw $a0, 0($sp)
111.         lw $ra, 4($sp)
112.         addi $sp, $sp, 8
113.         mul $v0, $a0, $v0
114.         jr $ra
```

?

```
1.   int fact (int n){
2.       if (n < 1)
3.           return(1);
4.       else
5.           return (n * fact(n-1));
6.   }
```

Registradores

End.	Dado
$sp	60
$ra	**?**
$a0	3
$t0	1
$v0	**6**



Memória Principal

End.	Instruções e Dados
0	
4	
8	
12	
16	
20	
24	
28	~~0~~
32	~~110~~
36	~~1~~
40	~~110~~
44	~~2~~
48	~~110~~
52	~~3~~
56	~~?~~
60	X
64	X
68	X
72	X
76	X

$sp

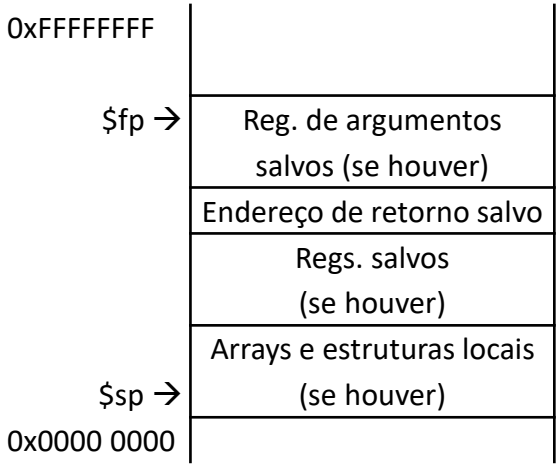
X: algum dado gravado.

Suporte para procedimentos

- Uma variável em C é um local na memória,:
  - Sua interpretação depende do seu tipo quanto da classe de armazenamento;
- A linguagem C possui duas classes de armazenamento:
  - Variáveis **Automáticas:** São locais a um procedimento:
    - São descartadas quando o procedimento termina;
  - Variáveis **estáticas:** Permanecem durante entradas e saídas de procedimento;
    - São variáveis **estáticas:**
      - Variáveis C declaradas fora de procedimentos **:**
      - Variáveis C declaradas dentro de procedimento com a palavra reservada `static`;
- O registrador de ponteiro global, `$gp` **:**
  - Simplificar o acesso aos **dados estáticos** por operações de `load` e `store`.

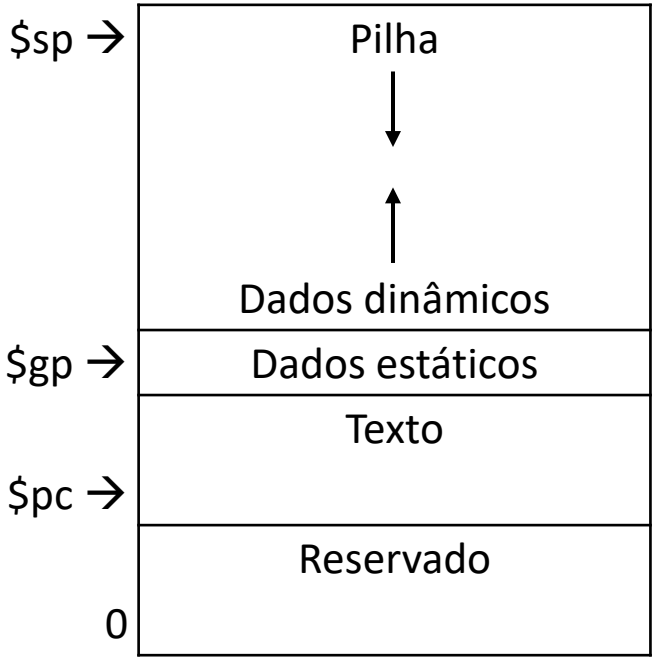
Suporte para procedimentos

- **Reservando espaço para novos dados na pilha:**
  - A pilha também é utilizada para armazenar variáveis que são locais ao procedimento que não cabem nos registradores, como arrays ou estruturas locais
  - O segmento da pilha que contém todos os registradores salvos e as variáveis locais de um procedimento é chamado de frame de procedimento ou registro de ativação.



Suporte para procedimentos

- **Reservando espaço para novos dados no *heap:***
  - Além de variáveis que são locais ao procedimentos, programadores precisam de espaço para variáveis e para estrutura de dados dinâmicas.



Suporte para procedimentos

- **Reservando espaço para novos dados no *heap:***
  - A pilha e o ***heap*** cresçam um em direção ao outro
    - Permite o uso eficiente da memória enquanto os dois segmentos aumentam e diminuem.
  - A linguagem C aloca e libera espaço no ***heap*** com funções explícitas:
    - `malloc()`: aloca espaço no *heap*.
    - `free()`: libera espaço no *heap*.

Assembly do MIPS

Categoria	Instrução	Exemplo	Significado
Aritmética	Add	`add $s1, $s2, $s3`	$s1 = $s2 + $s3
	Subtract	`sub $s1, $s2, $s3`	$s1 = $s2 - $s3
	Add immediate	`addi $s1, $s2, 4`	$s1 = $s2 - 4
Transf. de dados	Load word	`lw $s1, 100($s2)`	$s1 = Memória[$s2 + 100]
	Store word	`sw $s1, 100($s2)`	Memória[$s2 + 100] = $s1
Lógica	And	`and $s1, $s2, $s3`	$s1 = $s2 & $s3
	Or	`or $s1, $s2, $s3`	$s1 = $s2 | $s3
	Nor	`nor $s1, $s2, $s3`	$s1 = ~($s2 | $s3)
	And immediate	`andi $s1, $s2, 100`	$s1 = $s2 & 100
	Or immediate	`ori $s1, $s2, 100`	$s1 = $s2 | 100
	Shift left logical	`sll $s1, $s2, 10`	$s1 = $s2 << 10
	Shift righ logical	`srl $s1, $s2, 10`	$s1 = $s2 >> 10
Desvio condicional	Branch on equal	`beq $s1, $s2, L`	If ($s1 == $s2) go to L
	Branch on not equal	`bnq $s1, $s2, L`	If ($s1 != $s2) go to L
	Set on less than	`slt $s1, $s2, $s3`	If ($s2 < $s3) $s1 = 1 else $s1 = 0
	Set on less than immediate	`slti $s1, $s2, 100`	If ($s2 < 100) $s1 = 1 else $s1 = 0
Desvio incondicional	Jump	`j L`	Go to L
	Jump register	`jr $ra`	Go to $ra
	Jump and link	`jal L`	$ra = PC + 4 e go to L

COMUNICANDO-SE COM AS PESSOAS

Comunicando-se com as pessoas

- Representação de caracteres:
  - Código ASCII de 8 bits;
  - Computadores necessitam de instruções que realizem a movimentação de grupos de bits de palavras:

```
1.  lb $t0, 0($sp) # Load byte
2.  sb $t0, 0($sp) # Store byte
```

  - *Strings* são representados por conjuntos de caracteres, com geralmente três opções para representação:
    - A primeira posição da *string:*
      - Reservada para indicar o tamanho de uma *string*;
    - Uma variável acompanhante possui o tamanho da *string:*
      - Como uma estrutura;
    - A última posição da *string:*
      - Caractere que serve para marcar o final da *string* (‘\0’ na linguagem C).

Comunicando-se com as pessoas

- Compilando um procedimento de cópia de string para demonstrar o uso de strings em C

```
1.  void strcpy(char x[], char y[]){
2.      int i;
3.      i = 0;
4.      while ( (x[i] = y[i]) != '\0')
5.          i += 1;
6.  }
```

- Considere que os endereços base para os arrays `x` e `y` são encontrados em `$a0` e `$a1`

Comunicando-se com as pessoas

```
1.   void strcpy(char x[], char y[]){
2.       int i;
3.       i = 0;
4.       while ( (x[i] = y[i]) != '\0')
5.           i += 1;
6.   }
```

```
1.   strcpy: addi $sp, $sp, -4
2.           sw   $s0, 0($sp)
3.           add  $s0, $zero, $zero
4.   L1:     add  $t1, $s0, $a1  # endereço de y[i] em $t1
5.           lb   $t2, 0($t1)    # $t2 = y[i], como é um byte, não i*4
6.           add  $t3, $s0, $a0  # endereço de x[i] em $t3
7.           sb   $t2, 0($t3)    # x[i] = y[i]
8.           beq  $t2, $zero, L2 # se y[i]== 0 , vai para L2
9.           addi $s0, $s0, 1    # i = i + 1
10.          j    L1
11.  L2:     lw   $s0, 0 ($sp)
12.          addi $sp, $sp, 4
13.          jr   $ra
```

Comunicando-se com as pessoas

- Caracteres e strings em Java:
  - **Unicode** é uma codificação universal dos alfabetos da maior parte das linguagens humanas;
  - 16 bits para representar um caractere:

```
1.  lh $t0, 0($sp) # Load half-word
2.  sh $t0, 0($sp) # Store half-word
```

  - Ao contrário da linguagem C, Java reserva uma palavra para indicar o tamanho da string.

Comunicando-se com as pessoas

```
1.    void strcpy(char x[], char y[]){ // Unicode (16 bits)
2.        int i;
3.        i = 0;
4.        while ( (x[i] = y[i]) != '\0')
5.            i += 1;
6.    }
```

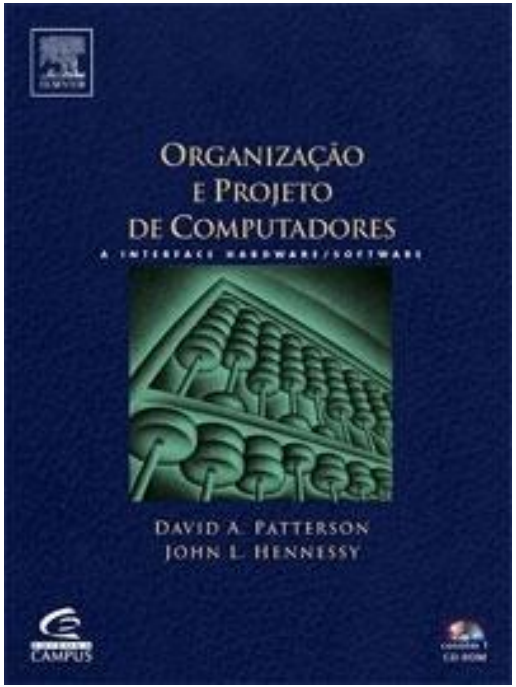
```
1.    strcpy: addi $sp, $sp, -4
2.            sw   $s0, 0($sp)
3.            add  $s0, $zero, $zero
4.    L1:     sll  $t0, $s0, 1     # $t0 = i * 2 (*)
5.            add  $t1, $t0, $a1   # endereço de y[i] em $t1
6.            lh   $t2, 0($t1)     # $t2 = y[i], como é um byte, não i*4
7.            add  $t3, $t0, $a0   # endereço de x[i] em $t3
8.            sh   $t2, 0($t3)     # x[i] = y[i]
9.            beq  $t2, $zero, L2 # se y[i]== 0 , vai para L2
10.           addi $s0, $s0, 1     # i = i + 1
11.           j    L1
12.   L2:     lw   $s0, 0 ($sp)
13.           addi $sp, $sp, 4
14.           jr   $ra
```

Assembly do MIPS

Categoria	Instrução	Exemplo	Significado
Aritmética	Add	`add $s1, $s2, $s3`	$s1 = $s2 + $s3
	Subtract	`sub $s1, $s2, $s3`	$s1 = $s2 - $s3
	Add immediate	`addi $s1, $s2, 4`	$s1 = $s2 - 4
Transf. de dados	Load word	`lw $s1, 100($s2)`	$s1 = Memória[$s2 + 100]
	Store word	`sw $s1, 100($s2)`	Memória[$s2 + 100] = $s1
	Load half	`lh $s1, 100($s2)`	$s1 = Memória[$s2 + 100]
	Store half	`sh $s1, 100($s2)`	Memória[$s2 + 100] = $s1
	Load byte	`lb $s1, 100($s2)`	$s1 = Memória[$s2 + 100]
	Store byte	`sb $s1, 100($s2)`	Memória[$s2 + 100] = $s1
Lógica	And	`and $s1, $s2, $s3`	$s1 = $s2 & $s3
	Or	`or $s1, $s2, $s3`	$s1 = $s2 | $s3
	Nor	`nor $s1, $s2, $s3`	$s1 = ~($s2 | $s3)
	And immediate	`andi $s1, $s2, 100`	$s1 = $s2 & 100
	Or immediate	`ori $s1, $s2, 100`	$s1 = $s2 | 100
	Shift left logical	`sll $s1, $s2, 10`	$s1 = $s2 << 10
	Shift righ logical	`srl $s1, $s2, 10`	$s1 = $s2 >> 10
Desvio condicional	Branch on equal	`beq $s1, $s2, L`	If ($s1 == $s2) go to L
	Branch on not equal	`bnq $s1, $s2, L`	If ($s1 != $s2) go to L
	Set on less than	`slt $s1, $s2, $s3`	If ($s2 < $s3) $s1 = 1 else $s1 = 0
	Set on less than immediate	`slti $s1, $s2, 100`	If ($s2 < 100) $s1 = 1 else $s1 = 0
Desvio incondicional	Jump	`j L`	Go to L
	Jump register	`jr $ra`	Go to $ra
	Jump and link	`jal L`	$ra = PC + 4 e go to L

Bibliografia

- PATTERSON, D. A.; HENNESSY, J. L. **Organização e Arquitetura de computadores: a interface hardware/software**. 3. ed. Rio de Janeiro: Elsevier, 2005.
  - CAPÍTULO 1



FIM